

课程号: 180206081100M1001H-01

第15章(第2讲)

支持向量机与核方法

Support Vector Machine & Kernel Methods

向 世 明

smxiang@nlpr.ia.ac.cn

<https://people.ucas.ac.cn/~xiangshiming>

时空数据分析与学习课题组 (STDAL)

中科院自动化研究所 多模态人工智能系统国家重点实验室

助教: 杨 奇 (yangqi2021@ia.ac.cn)

张 涛 (zhangtao2021@ia.ac.cn)

上次课核心知识点

• VC维

- 假设空间 H 的VC维是能被 H 打散的最大示例集的大小。
- $VC(H)=d$ 表明存在大小为 d 的示例集能被假设空间 H 打散。
- 通常按如下方式来计算VC维：
 - 若存在大小为 d 的示例集能被 H 打散，但不存在任何大小为 $d+1$ 示例集能被 H 打散，则 H 的VC维是 d 。

The Probabilistic Guarantee:

$$E_{test} \leq E_{train} + \left(\frac{h + h \log(2N/h) - \log(p/4)}{N} \right)^{\frac{1}{2}}$$

置信水平: $\log(1/\delta)$

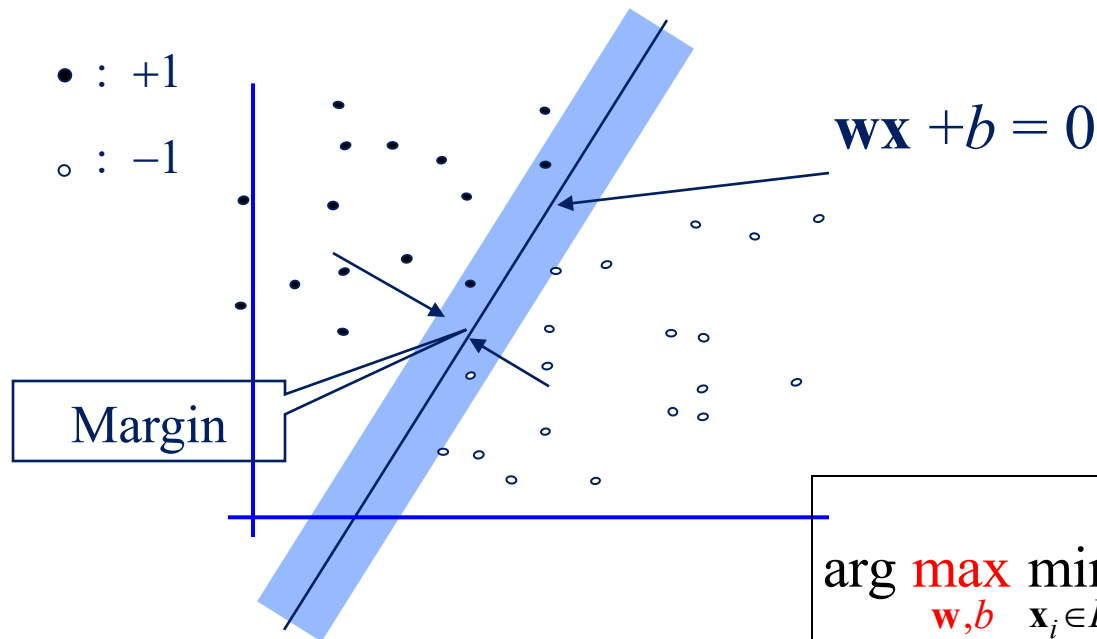
where N = size of training set

h = VC dimension of the model

p = upper bound on probability that this bound fails

此边界失败的概率上限

Estimate Margin (Method 1)



$$\mathbf{w}\mathbf{x}_i + b \geq 0 \text{ iff } y_i = 1$$

$$\mathbf{w}\mathbf{x}_i + b \leq 0 \text{ iff } y_i = -1$$



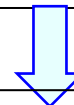
$$y_i(\mathbf{w}\mathbf{x}_i + b) \geq 0$$

Strategy:

$$\forall \mathbf{x}_i \in D: |b + \mathbf{x}_i \cdot \mathbf{w}| \geq 1$$

$$\arg \max_{\mathbf{w}, b} \min_{\mathbf{x}_i \in D} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}}$$

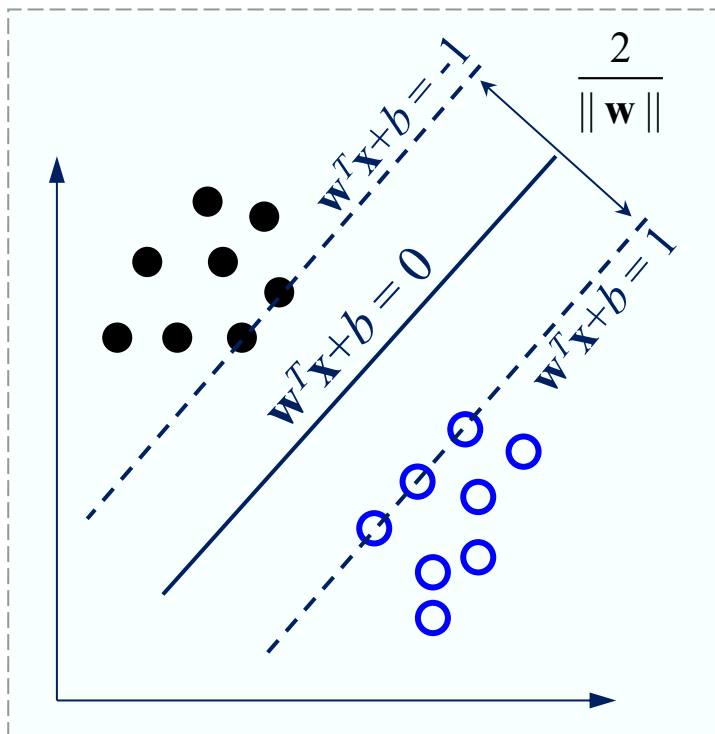
subject to $\forall \mathbf{x}_i \in D: y_i(\mathbf{x}_i \cdot \mathbf{w} + b) \geq 0$



$$\arg \min_{\mathbf{w}, b} \sum_{i=1}^d w_i^2$$

subject to $\forall \mathbf{x}_i \in D: y_i(\mathbf{x}_i \cdot \mathbf{w} + b) \geq 1$

Estimate Margin (Method 1)



- ✓ 让负类样本点 $\mathbf{w}^T \mathbf{x} + b \leq -1$
- ✓ 让正类样本点 $\mathbf{w}^T \mathbf{x} + b \geq +1$
- ✓ 注意到负类标签 $y_i = -1$, 正类标签 $y_i = +1$, 综合起来, 有:

$$y_i (\mathbf{w}^T \mathbf{x} + b) \geq 1$$

总是可以做到的 (其一: 数值上)

Strategy:

$$\forall \mathbf{x}_i \in D: |b + \mathbf{x}_i \cdot \mathbf{w}| \geq 1$$

$$\begin{aligned} & \arg \max_{\mathbf{w}, b} \min_{\mathbf{x}_i \in D} \frac{|b + \mathbf{x}_i \cdot \mathbf{w}|}{\sqrt{\sum_{i=1}^d w_i^2}} \\ & \text{subject to } \forall \mathbf{x}_i \in D: y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 0 \end{aligned}$$



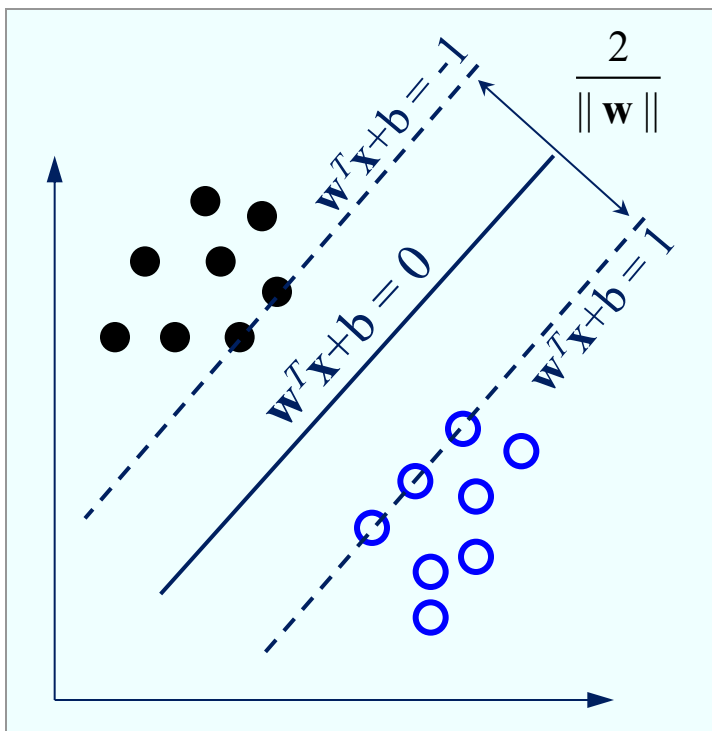
$$\begin{aligned} & \arg \min_{\mathbf{w}, b} \sum_{i=1}^d w_i^2 \\ & \text{subject to } \forall \mathbf{x}_i \in D: y_i (\mathbf{x}_i \cdot \mathbf{w} + b) \geq 1 \end{aligned}$$

(其二, 考虑类别时的约束统一表示)

线性可分支持向量机

- 学习模型

线性可分情形



给定训练集:

$$T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\}, y_i \in \{+1, -1\}$$

任务: 估计最大间隔分类超平面

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

分类超平面: $\mathbf{w}^T \mathbf{x} + b = 0$

分类决策函数: $f(\mathbf{x}) = \text{sign}(\mathbf{w}^T \mathbf{x} + b)$

线性不可分-学习模型（支持向量机）

Describe the Theory

Describe the Mistake

体现了表达能力

体现了经验风险

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

C : tradeoff parameter between error and margin;
chosen by the user;
large C means a higher penalty to errors

目标函数第一项表示使margin尽量大，第二项表示使误差分类点的个数尽量小。

• 复习KKT条件 (数学知识点, 不要求):

KKT条件

原始问题

$$\begin{aligned} \min_{\mathbf{x} \in R^d} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & c_i(\mathbf{x}) \leq 0, \\ & i = 1, 2, \dots, k \\ & h_j(\mathbf{x}) = 0, \\ & j = 1, 2, \dots, l \end{aligned}$$

$$\nabla_{\mathbf{x}} L(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = 0$$

$$\nabla_{\boldsymbol{\alpha}} L(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = 0$$

$$\nabla_{\boldsymbol{\beta}} L(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = 0$$

$$\alpha_i c_i(\mathbf{x}) = 0, \quad i = 1, 2, \dots, k$$

$$c_i(\mathbf{x}) \leq 0, \quad i = 1, 2, \dots, k$$

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, k$$

$$h_j(\mathbf{x}) = 0, \quad j = 1, 2, \dots, l$$

$$L(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = f(\mathbf{x}) + \sum_{i=1}^k \alpha_i c_i(\mathbf{x}) + \sum_{j=1}^l \beta_j h_j(\mathbf{x})$$

广义拉格朗日函数

支持向量机(对偶)

• 对偶算法 (线性可分情形)

- ✓ 在约束最优化问题中, 常利用拉格朗日对偶性将原始问题转化为对偶问题进行求解
- ✓ 对偶算法往往容易求解
- ✓ 对偶算法可以推广到核学习

$$\min_{\mathbf{x} \in R^d} f(\mathbf{x})$$

$$s.t. \quad c_i(\mathbf{x}) \leq 0, \quad i = 1, 2, \dots, k$$

$$h_j(\mathbf{x}) = 0, \quad j = 1, 2, \dots, l$$

广义拉格朗日函数

$$L(\mathbf{x}, \boldsymbol{\alpha}, \boldsymbol{\beta}) = f(\mathbf{x}) + \sum_{i=1}^k \alpha_i c_i(\mathbf{x}) + \sum_{j=1}^l \beta_j h_j(\mathbf{x})$$

数学知识点

(拉格朗日对偶性) $\Rightarrow$

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ s.t. \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

原始问题

$$\max_{\boldsymbol{\alpha}} \min_{\mathbf{w}, b} \quad L(\mathbf{w}, b, \boldsymbol{\alpha})$$

$$s.t. \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, n$$

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^n \alpha_i y_i(\mathbf{w}^T \mathbf{x}_i + b) + \sum_{i=1}^n \alpha_i$$

对偶问题

证明见: 李航: 统计学习方法, 清华大学出版社, 2012 (第7章) (本课程不要求)

支持向量机(对偶)

- 对偶问题求解 (线性可分)

– (1) 求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\alpha})$

$$\nabla_{\mathbf{w}} L(\mathbf{w}, b, \boldsymbol{\alpha}) = \mathbf{w} - \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i = \mathbf{0} \Rightarrow \mathbf{w} = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\nabla_b L(\mathbf{w}, b, \boldsymbol{\alpha}) = \sum_{i=1}^n \alpha_i y_i = 0 \Rightarrow \sum_{i=1}^n \alpha_i y_i = 0$$



$$\min_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\alpha}) = -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

支持向量机(对偶)

– (2) 求对偶问题, 即求 $\min_{\mathbf{w}, b} L(\mathbf{w}, b, \boldsymbol{\alpha})$ 对 $\boldsymbol{\alpha}$ 的极大

$$\max_{\boldsymbol{\alpha}} \quad -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

$$s.t. \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, n$$

✓ This is a convex quadratic programming (QP) problem

✓ Global maximum of α_i can always be found

➤ well established tools for solving this optimization problem

$$\min_{\boldsymbol{\alpha}} \quad \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \sum_{i=1}^n \alpha_i$$

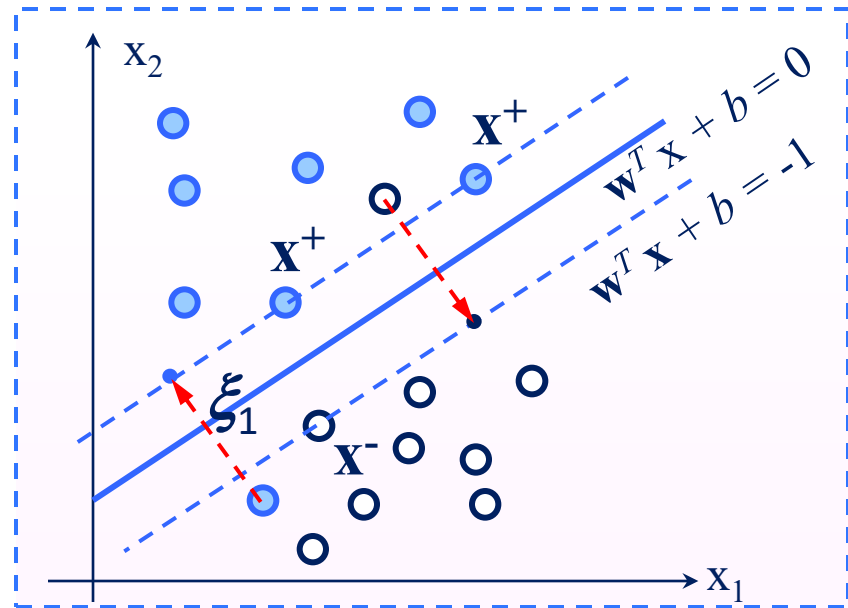
$$s.t. \quad \sum_{i=1}^n \alpha_i y_i = 0$$

$$\alpha_i \geq 0, \quad i = 1, 2, \dots, n$$



软间隔最大化（线性不可分）：

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned} \quad \text{原始问题}$$



↓ (广义拉格朗日函数)

↓

$$L(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i - \sum_{i=1}^n \alpha_i y_i (\mathbf{w}^T \mathbf{x}_i + b + \xi_i) + \sum_{i=1}^n \alpha_i - \sum_{i=1}^n \mu_i \xi_i$$

↓

拉格朗日对偶

$$\max_{\substack{\alpha \geq 0 \\ w, b, \xi \\ \mu \geq 0}} \min L(w, b, \xi, \alpha, \mu)$$

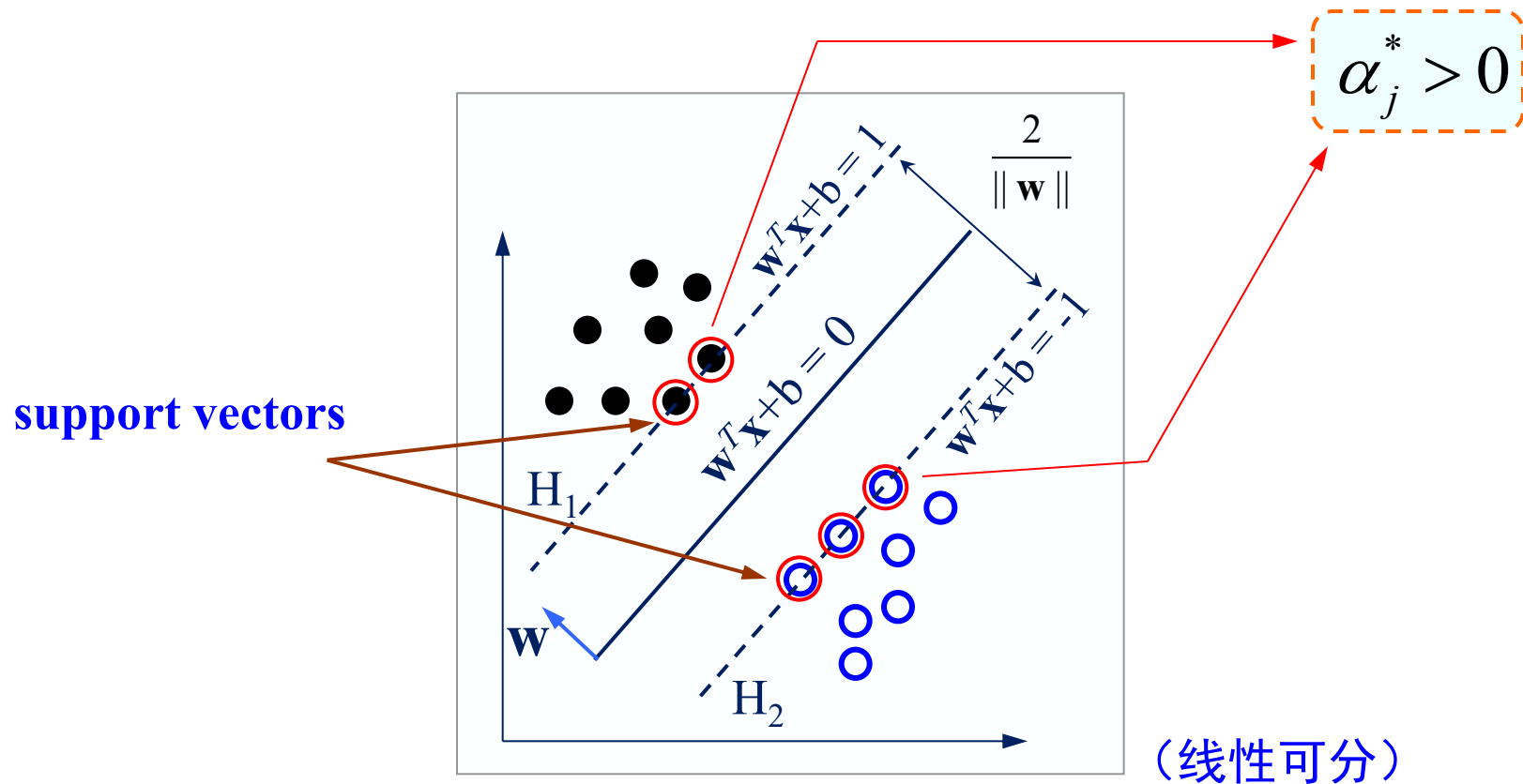
$$\min_{\alpha} \quad \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \sum_{i=1}^n \alpha_i$$

$$\text{s.t.} \quad \sum_{i=1}^n \alpha_i y_i = 0;$$

对偶问题

$$0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, n$$

✓ 使等式成立的点为**支持向量**: $y_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$



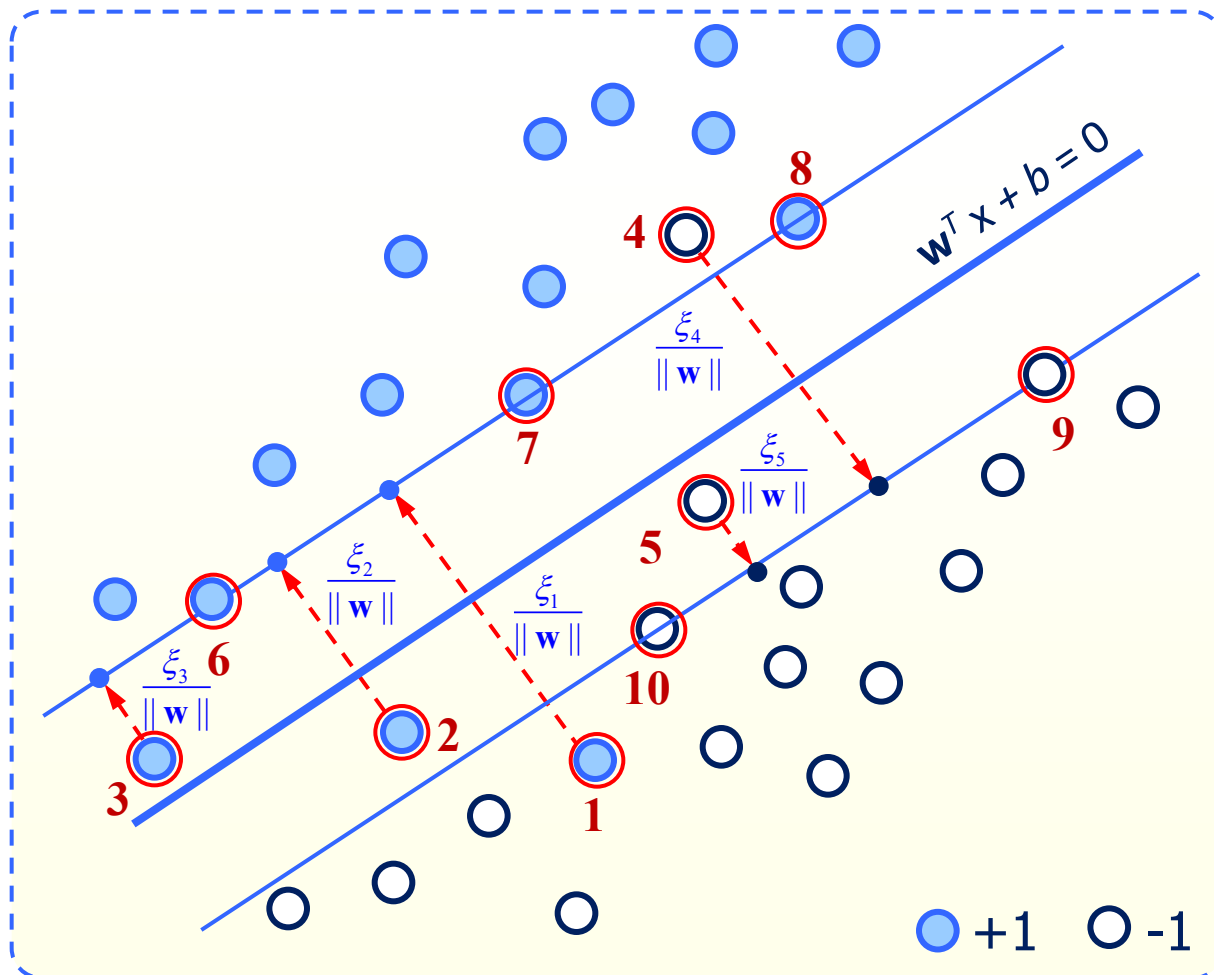
所有样本中，“支持向量”到分类面的几何距离最小。

软间隔支持向量：注意 $\alpha_j^* > 0$ 的样本点均称为支持向量！

图中，第一个点到其正确边界的距离为： $\frac{\xi_1}{\|\mathbf{w}\|}$ ，其它类推。

- 1: $\alpha^* = C, \xi_1 > 1$
- 2: $\alpha^* = C, \xi_2 > 1$
- 3: $\alpha^* = C, 0 < \xi_3 < 1$
- 4: $\alpha^* = C, \xi_4 > 1$
- 5: $\alpha^* = C, 0 < \xi_5 < 1$
- 6: $0 < \alpha^* < C, \xi_6 = 0$
- 7: $0 < \alpha^* < C, \xi_7 = 0$
- 8: $0 < \alpha^* < C, \xi_8 = 0$
- 9: $0 < \alpha^* < C, \xi_9 = 0$
- 10: $0 < \alpha^* < C, \xi_{10} = 0$

10个支持向量



图中带红色圆圈的均表示支持向量

15.3.3 支持向量机解的存在性

- 定理3

- 设 $\boldsymbol{\alpha}^* = [\alpha_1^*, \alpha_1^*, \dots, \alpha_n^*]^T \in R^n$ 是对偶问题的解，则至少存在一个下标 j ，有 $0 < \alpha_j^* < C$ ，可按下式求得原始问题的最优解：

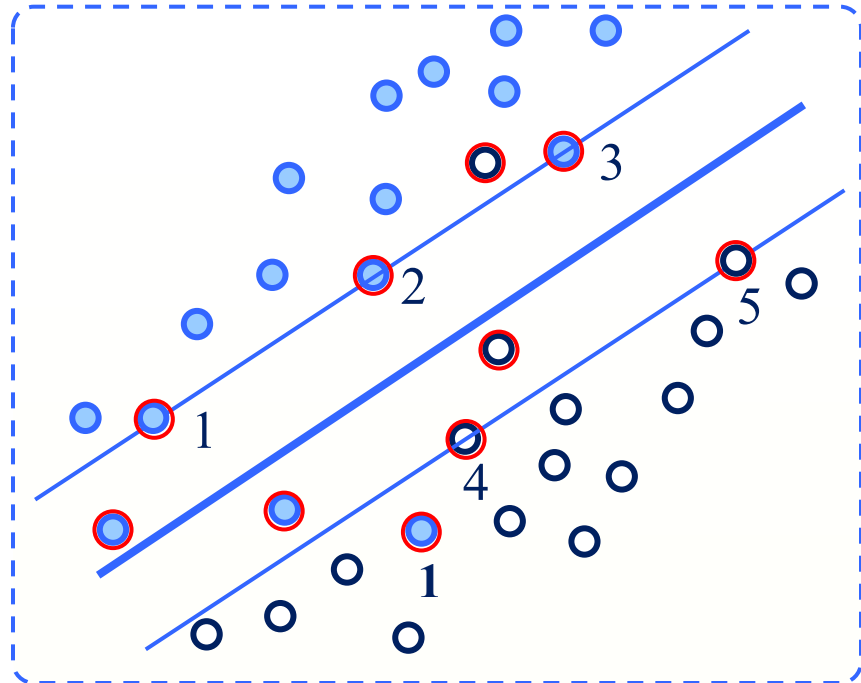
$$\mathbf{w}^* = \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i, \quad b^* = y_j - \sum_{i=1}^n \alpha_i^* y_i (\mathbf{x}_i \cdot \mathbf{x}_j)$$

- 分类超平面： $\mathbf{w}^* \cdot \mathbf{x} + b^* = 0$

- 分类决策函数： $f(\mathbf{x}) = \text{sign}(\mathbf{w}^* \cdot \mathbf{x} + b^*) = \text{sign}\left(\sum_{i=1}^n \alpha_i^* y_i (\mathbf{x} \cdot \mathbf{x}_i) + b^*\right)$

15.3.3 支持向量机解的存在性

- 偏置 b 的确定
 - 不唯一



$$b^* = y_j - \sum_{i=1}^n y_i \alpha_i^* (\mathbf{x}_i \cdot \mathbf{x}_j), \quad 0 < \alpha_j^* < C$$

在所有符合条件的样本上计算一个 b^* ，然后取平均：

编程技巧：

$$b^* = \frac{1}{|\{\alpha_k^* : 0 < \alpha_k^* < C\}|} \sum_{0 < \alpha_k^* < C} (y_k - \sum_{i=1}^n \alpha_i^* y_i \mathbf{x}_i \cdot \mathbf{x}_j)$$

$|\{\cdot\}|$: 表示集合的基，也就是集合元素的个数

KSVM:

- 从对偶问题直接实现SVM核化—训练

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & C \geq \alpha_i \geq 0, \quad i = 1, 2, \dots, n \end{aligned}$$



$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0 \\ & C \geq \alpha_i \geq 0, \quad i = 1, 2, \dots, n \end{aligned}$$

KSVM:

- 预测 (对新数据)

$$f(\mathbf{x}) = \text{sign} \left(\sum_{i=1}^n \alpha_i^* y_i (\mathbf{x} \cdot \mathbf{x}_i) + b^* \right), \quad b^* = y_j - \sum_{i=1}^n \alpha_i^* y_i (\mathbf{x}_i \cdot \mathbf{x}_j)$$



$$f(\mathbf{x}) = \text{sign} \left(\sum_{i=1}^n \alpha_i^* y_i K(\mathbf{x} \cdot \mathbf{x}_i) + b^* \right), \quad b^* = y_j - \sum_{i=1}^n \alpha_i^* y_i K(\mathbf{x}_i \cdot \mathbf{x}_j)$$

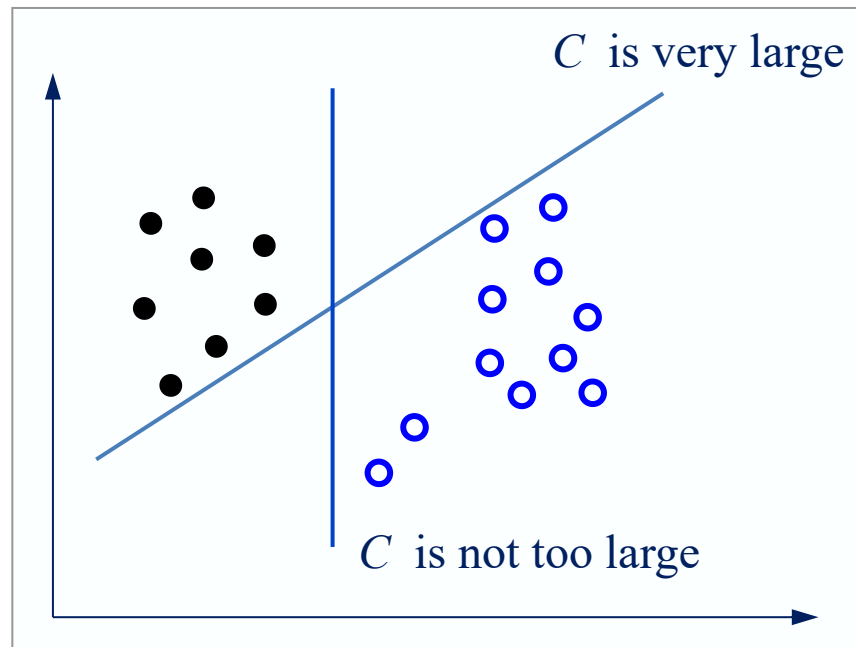
15.7 Model Selection

15.7 模型选择

The “C” Problem:

- “C” plays a major role in controlling “over-fitting”
- Finding the “Right” value for “C” is one of the major problems of SVM

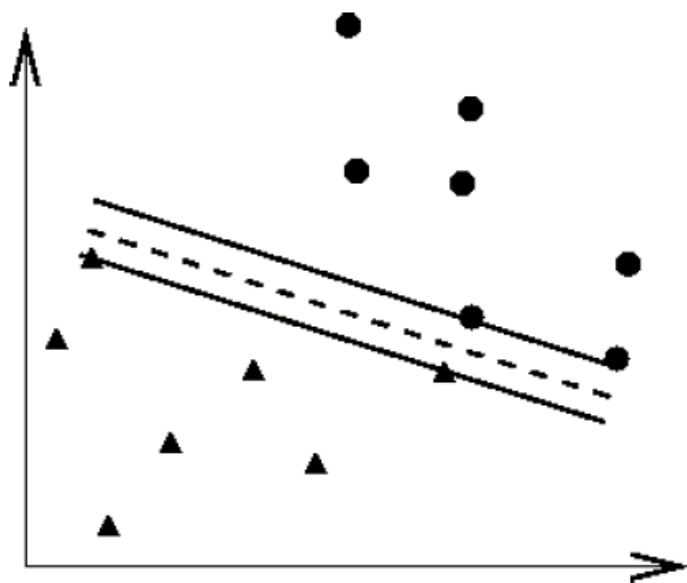
$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$



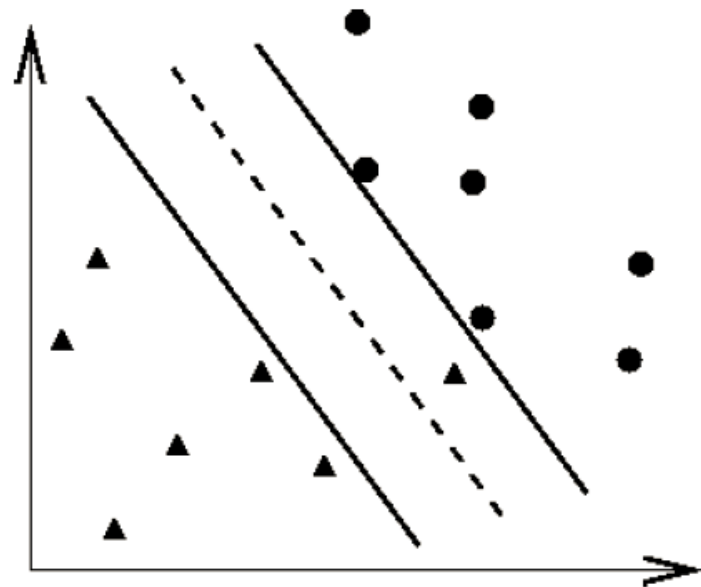
15.7 模型选择

不同 C 值对分类面的影响:

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$



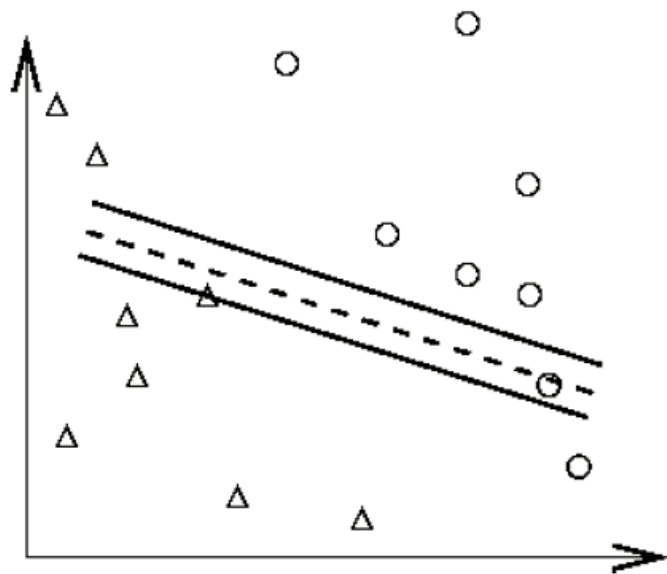
C 值较大, 更加关心错分样本,
倾向于产生没有错分样本的
分界面



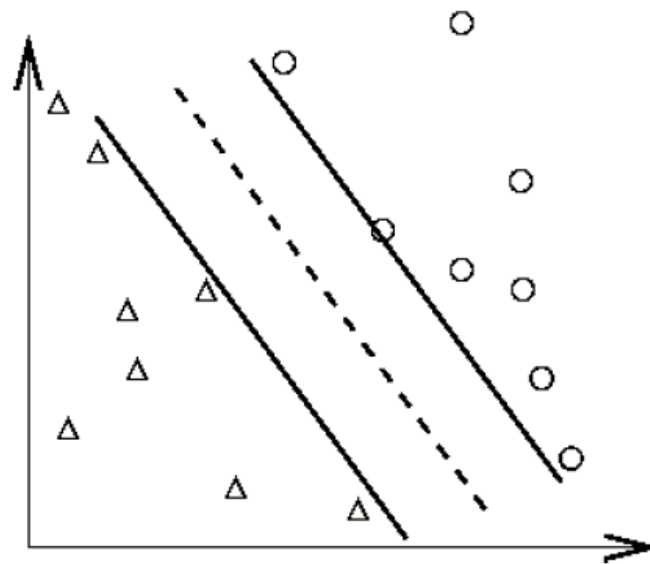
C 值较小, 更加关心分类间
隔, 倾向于产生大间隔的分
界面

15.7 模型选择

通过选择合适的C值，适当地平衡分类间隔，能减少过拟合风险。



过拟合 (overfitting)



泛化性更好

15.7 模型选择

C and Kernel:

- In practice, a **Gaussian radial basis** or a **low degree polynomial kernel** is a good start
- Checking which set of parameters (such as C or σ if we choose RBF kernel) are the most appropriate by Cross-Validation (K-fold): (K-折交叉验证)
 - divide randomly all the available training examples into K equal-sized subsets
 - use all but one subset to train the SVM with the chosen parameters
 - use the held-out subset to measure classification error
 - repeat above two steps for each subset
 - average the results to get an estimate of the generalization error of the SVM classifier

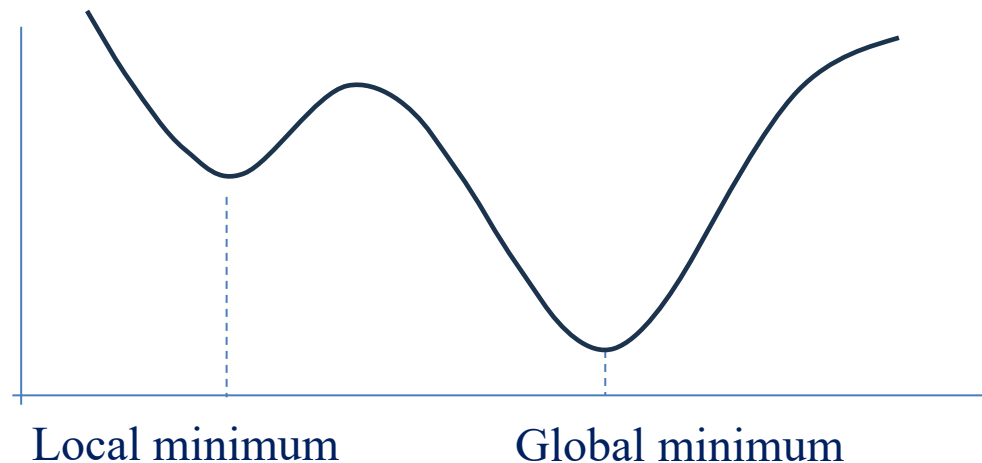
Why cross-validation?

15.8 Optimization

15.8.1 回顾最优化方法

Local and Global Optimal

- Unique solution?
- Depend on initialization?



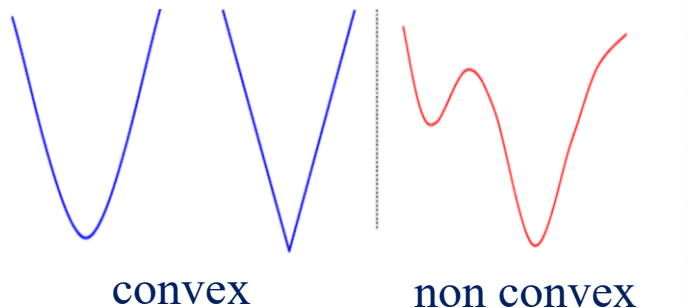
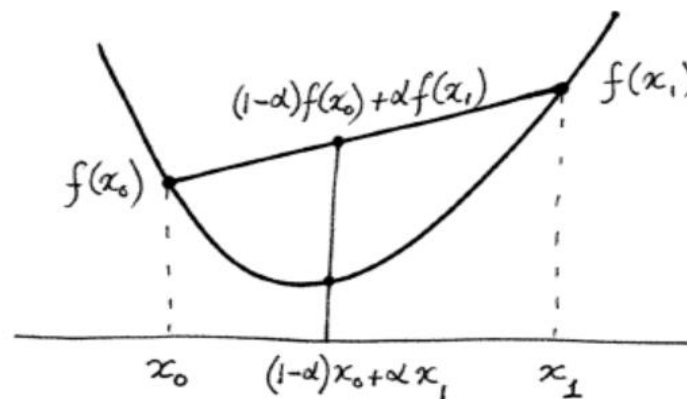
- If the cost function is convex, then a locally optimal point is globally optimal

Convex Function:

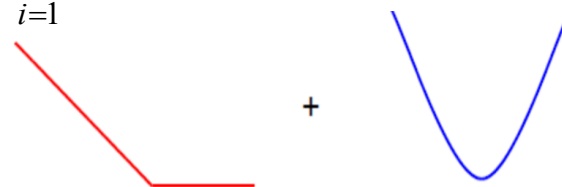
Give a domain D in R^d , a convex function $f: D \rightarrow R$ is one that satisfies, for any $\mathbf{x}_0$ and $\mathbf{x}_1$ in D :

$$f((1-\alpha)\mathbf{x}_0 + \alpha\mathbf{x}_1) \leq (1-\alpha)f(\mathbf{x}_0) + \alpha f(\mathbf{x}_1)$$

Line joining $(\mathbf{x}_0, f(\mathbf{x}_0))$ and $(\mathbf{x}_1, f(\mathbf{x}_1))$ lies above the function (curve or surface)



$$\min_{\mathbf{w}, b} \sum_{i=1}^n [1 - y_i(\mathbf{w}^T \mathbf{x}_i + b)]_+ + \lambda \|\mathbf{w}\|^2$$



SVM is convex: A non-negative sum of convex functions is convex.

15.8.1 回顾最优化方法

二次规划 (Quadratic Programming, QP)

- 二次规划是非线性规划的一个核心分支，其核心特征是目标函数为二次函数，约束条件为线性等式或不等式

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{H} \mathbf{x} + \mathbf{c}^T \mathbf{x} \\ \text{s.t.} \quad & \mathbf{A} \mathbf{x} \leq \mathbf{b}, \\ & \mathbf{B} \mathbf{x} = \mathbf{y}, \\ & \mathbf{l} \leq \mathbf{x} \leq \mathbf{u} \end{aligned}$$

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

- 有效集 (Active Set) 方法
- 内点法 (Interior Point Method)
- 梯度投影法

- 支持向量机是一个凸二次规划问题

Quadratic Programming:

- Many approaches have been developed

https://en.wikipedia.org/wiki/Quadratic_programming

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

Name	Brief info
AIMMS	A software system for modeling and solving optimization and scheduling-type problems
ALGLIB	Dual licensed (GPL/proprietary) numerical library (C++, .NET).
AMPL	A popular modeling language for large-scale mathematical optimization.
APMonitor	Modeling and optimization suite for LP, QP, NLP, MILP, MINLP, and DAE systems in MATLAB and Python.
Artelys Knitro	An Integrated Package for Nonlinear Optimization
CGAL	An open source computational geometry package which includes a quadratic programming solver.
CPLEX	Popular solver with an API (C, C++, Java, .Net, Python, Matlab and R). Free for academics.
Excel Solver Function	A nonlinear solver adjusted to spreadsheets in which function evaluations are based on the recalculating cells. Basic version available as a standard add-on for Excel.
GAMS	A high-level modeling system for mathematical optimization
GNU Octave	A free (its licence is GPLv3) general-purpose and matrix-oriented programming-language for numerical computing, similar to MATLAB. Quadratic programming in GNU Octave is available via its <code>qp</code> command
HiGHS	Open-source software for solving linear programming (LP), mixed-integer programming (MIP), and convex quadratic programming (QP) models
IMSL	A set of mathematical and statistical functions that programmers can embed into their software applications.

Quadratic Programming

- Most are “interior-point” methods: 内点法
 - 二次规划的内点法是一种用于解决具有等式和不等式约束的优化问题的方法。
 - 内点法通过将约束条件纳入目标函数，逐步求解，从而避免了对可行域边界的处理。
- For SVM, sequential minimal optimization (SMO, 序列最小最优化算法, 1998, by Platt) seems to be the most popular:
 - A QP with two variables is trivial to solve (求解起来很简单)
 - Each iteration of SMO picks a pair of (α_i, α_j) and solve the QP with these two variables; repeat until convergence
- In practice, we can just regard the QP solver as a “blackbox” without bothering how it works
 - considering BP in deep learning

15.8.2 Speed up the training for SVM

- **Hint 1: Only SVs will influence the decision boundary**

$$\max_a \quad -\frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i \cdot \mathbf{x}_j) + \sum_{i=1}^n \alpha_i$$

$$s.t. \quad \boxed{\sum_{i=1}^n \alpha_i y_i = 0} \quad \text{-----} \rightarrow \quad \alpha_1 = -y_1 \sum_{i=2}^n \alpha_i y_i$$

$$C \geq \alpha_i \geq 0, \quad i = 1, 2, \dots, n$$

如果只有 (α_1, α_2) 两个变量待求，
在 α_2 确定的情况下，可得 α_1

- ✓ **Training SVM only on part data in each iteration**
- ✓ keeping remaining the SVs and repeating training by adding new data sequentially until SVs don't change
- ✓ 整个SMO算法包含两个部分（Chunk strategy，组块策略）：
 - 求解两个变量的二次规划的解析解
 - 选择变量的启发式方法

15.8.2 Speed up the training for SVM

- **Hint 2: When the optimum is achieved, KKT conditions are satisfied**

$$\begin{cases} \alpha_i = 0 & \Rightarrow y_i f(x_i) \geq 1 \Rightarrow \text{Samples outside the boundary} \\ 0 < \alpha_i < C & \Rightarrow y_i f(x_i) = 1 \Rightarrow \text{Samples on the boundary} \\ \alpha_i = C & \Rightarrow y_i f(x_i) \leq 1 \Rightarrow \text{Samples within the boundary} \end{cases}$$

- **Sequential Minimal Optimization (SMO): 序列最小最优化算法**
 - Each time, choose two samples **violating the KKT condition** and updating the weights until all the points satisfied KKT condition (每次选择两个)
 - J. C. Platt, *Fast Training of Support Vector Machines Using Sequential Minimal Optimization*, Advances in kernel methods: support vector learning, 1999.

SMO:

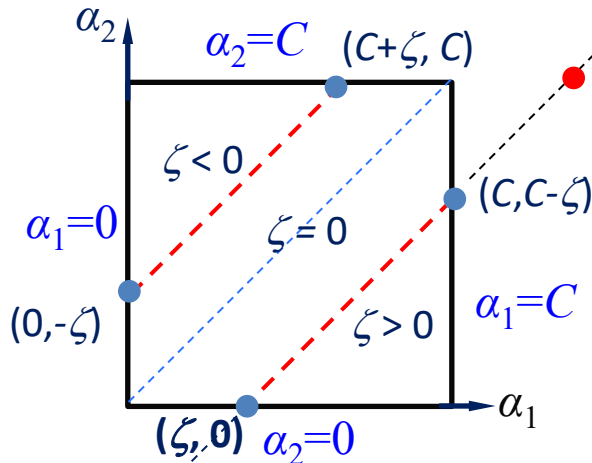
- Constraints on the Lagrange Multipliers (select two):

$$\min_{\alpha_1, \alpha_2} g(\alpha_1, \alpha_2) = \frac{1}{2} K_{11} \alpha_1^2 + \frac{1}{2} K_{22} \alpha_2^2 + y_1 y_2 K_{12} \alpha_1 \alpha_2$$

$$-(\alpha_1 + \alpha_2) + y_1 \alpha_1 \sum_{i=3}^n \alpha_i y_i K_{i1} + y_2 \alpha_2 \sum_{i=3}^n \alpha_i y_i K_{i2} + const$$

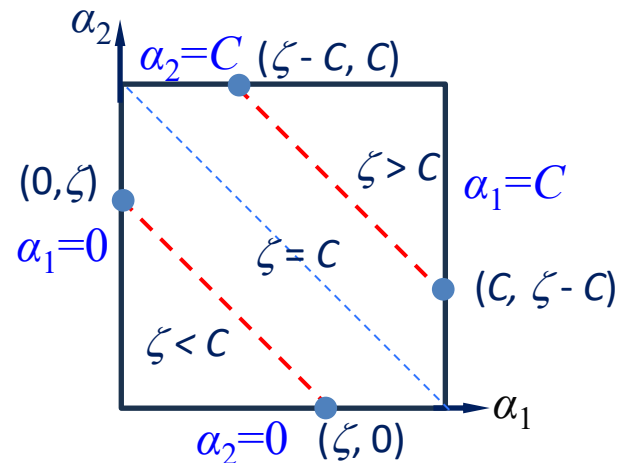
$$s.t. \quad y_1 \alpha_1 + y_2 \alpha_2 = -\sum_{i=3}^n \alpha_i y_i = \zeta$$

$$C \geq \alpha_i \geq 0, \quad i = 1, 2$$



比如，无约束条件时会在该条直接取得

$$y_1 \neq y_2 \Rightarrow \alpha_1 - \alpha_2 = \zeta$$



$$y_1 = y_2 \Rightarrow \alpha_1 + \alpha_2 = \zeta$$

细节求解：李航, 统计机器学习, 清华大学出版社（第一版），P. 124-131

SMO: 选取两个Lagrange乘子变量的启发式规则:

- 第一个Lagrange乘子 α_1 的选择:
 - 任何违反KKT条件的乘子均可作为第一个Lagrange乘子。
 - 第一个Lagrange乘子的选择构成SMO算法的外层循环。
 - 优先检查满足 $(0 < \alpha_i < C)$ 的样本（标准支持向量）
 - 检查违反KKT条件最严重的 $(y_i f(x_i))$ 与 1 偏差最大的样本，within ε , e.g. 10^{-3}).
 - The outer loop then goes back and iterates over the entire training set.
- 第二个Lagrange乘子 α_2 的选择
 - 选择目标函数下降最快的 α_2 :
 - 或者：与第一个乘子的结合，应该使第二个乘子的迭代步长较大（即使 α_2 的变化最大，即使 $|E_1 - E_2|$ 变化最大，其中 $E_i = f(x_i) - y_i$ ）,
- 很多文献改进方法：如何选择两个乘子变量
 - Maximal violating pair
 - Second order information

15.9 Multi-Class SVM

15.9.1 Multi-class Extension

- SVMs can only handle two-class outputs. (支持向量机只能处理两类分类问题)
- What can be done?
 - One answer: with output C labels, learn C SVM's: 一对多
 - SVM 1 learns “Output==1” vs “Output != 1”
 - SVM 2 learns “Output==2” vs “Output != 2”
 -
 - SVM C learns “Output==C” vs “Output != C”
 - 决策：在C个机器中选择最大的预测值对应的类作为其类别
- Then to predict the output for a new input, just predict with each SVM and find out which one puts the prediction the furthest into the positive region.

15.9.1 Multi-class Extension

Two families of methods for Multi-Class SVM:

- **Multiple Binary Problem**

- One-vs-All:

- C quadratic programming problems are solved

- One-vs-One:

- $C(C-1)/2$ quadratic programming (决策时: 投票)

- DAGSVM:

- $C(C-1)/2$ quadratic programming

- Half-vs-Half:

- 多个类别 (比如一半) 组成一组, 然后构建一棵DAG

- ECOC (Error-correcting output codes, 纠错输出码)

- 引入停用类

- LatticeSVM:

- Z. Liu, L. Jin, LatticeSVM—A New Method for Multi-Class Support Vector Machines, *IJCNN 2008*.

15.9.1 Multi-class Extension

Two families of methods for Multi-Class SVM:

- **Single Machine**

- Consider all classes at once, result in a much larger optimization problem in one step.
 - ✓ K. Crammer, Y. Singer, On the Algorithmic Implementation of Multiclass Kernel-based Vector Machines, *JMLR* 2001.
 - ✓ J. Weston, C. Watkins, Multi-Class Support Vector Machines, *Technical Report* 1998.
- C.-W. Hsu, C. Lin, A Comparison of Methods for Multiclass Support Vector Machines, *IEEE Trans. Neural Networks*, 2002.

15.9.2 DAGSVM

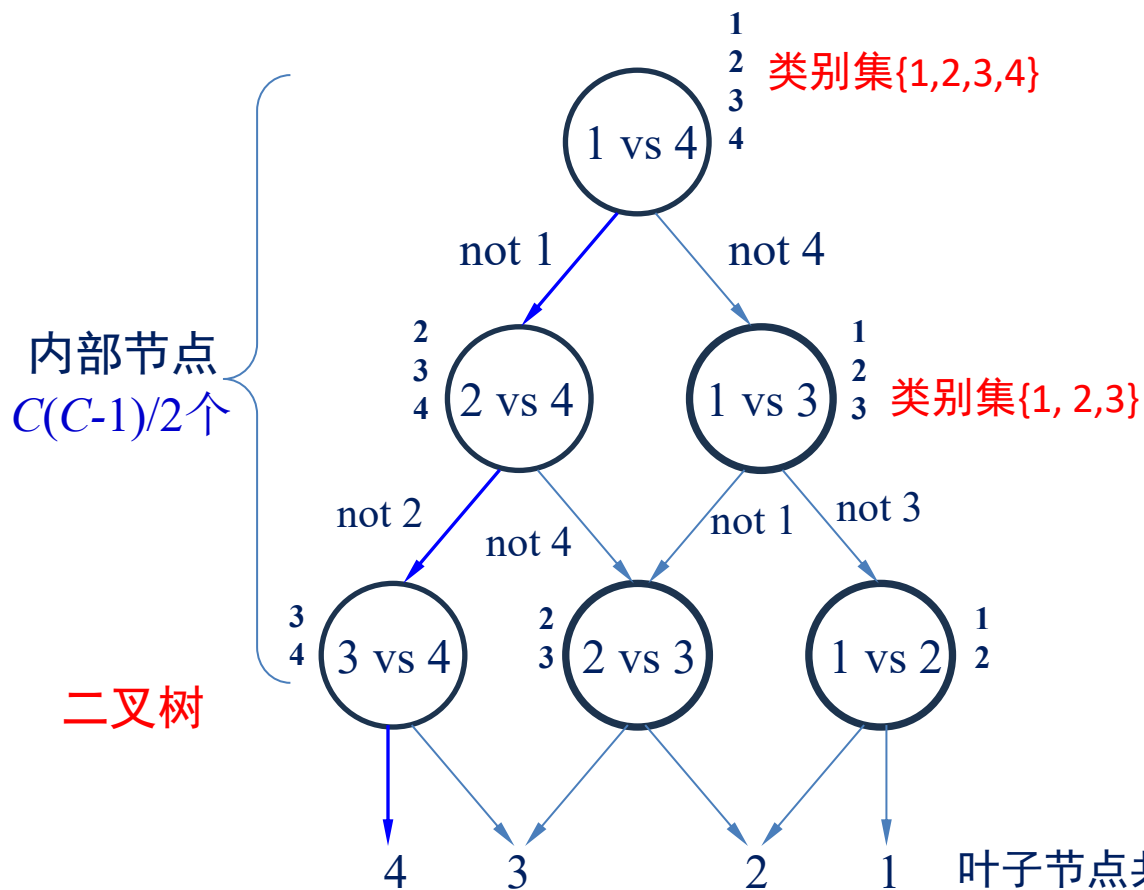
DAGSVM (**D**irected **A**cyclic **G**raph **S**upport **V**ector **M**achine) 通过构建一个**有向无环图 (DAG)** 来表示多个分类器，**每个节点代表一个二分类器**，节点之间的边表示类别的决策边界。可有效地将多类分类问题转化为一系列的二分类问题，从而简化问题的复杂度。

- 先训练 $C(C-1)/2$ 个两类支持向量机。
- 构建DAG图：对于有 C 个类别的分类问题，构建一个有 $C(C-1)/2$ 个节点的DAG图。每个节点代表一个二分类问题，节点之间的边表示类别之间的决策边界。**核心：1 VS 1**
- 训练二分类器：在每个节点上训练一个二分类器，将当前节点代表的两个类别分开。
- 预测：在进行预测时，从根节点开始，根据每个节点的分类结果逐步向下决策，直到到达叶子节点，最终确定样本的类别。
- **The choice of the class order in the list is arbitrary.**

J.C. Platt et al, Large Margin DAGs for Multiclass Classification, *NIPS 2000*.

15.9.2 DAGSVM

DAGSVM: 通过构建一个有向无环图（DAG）来表示多个分类器，每个节点代表一个二分类器，节点之间的边表示类别的决策边界。



: 根节点对应 “第一个类别 - 最后一个类别” 的类别对

第 j 层 (j 从 1 到 $C-1$) 有 j 个内部节点，第 i 个节点连接到第 $j+1$ 层的第 i 个和第 $i+1$ 个节点（形成三角形层级结构）

- ✓ 在训练阶段，仍然要构建 $C(C-1)/2$ 个 SVM
- ✓ 随着路径的增长，失败的类别逐渐被过滤掉
- ✓ 在测试阶段，则不必运行 $C(C-1)/2$ 个 SVM，沿树进行，仅需评估 $C-1$ 个节点，节省预测时间

J.C. Platt et al, Large Margin DAGs for Multiclass Classification, *NIPS 2000*.

15.9.2 DAGSVM

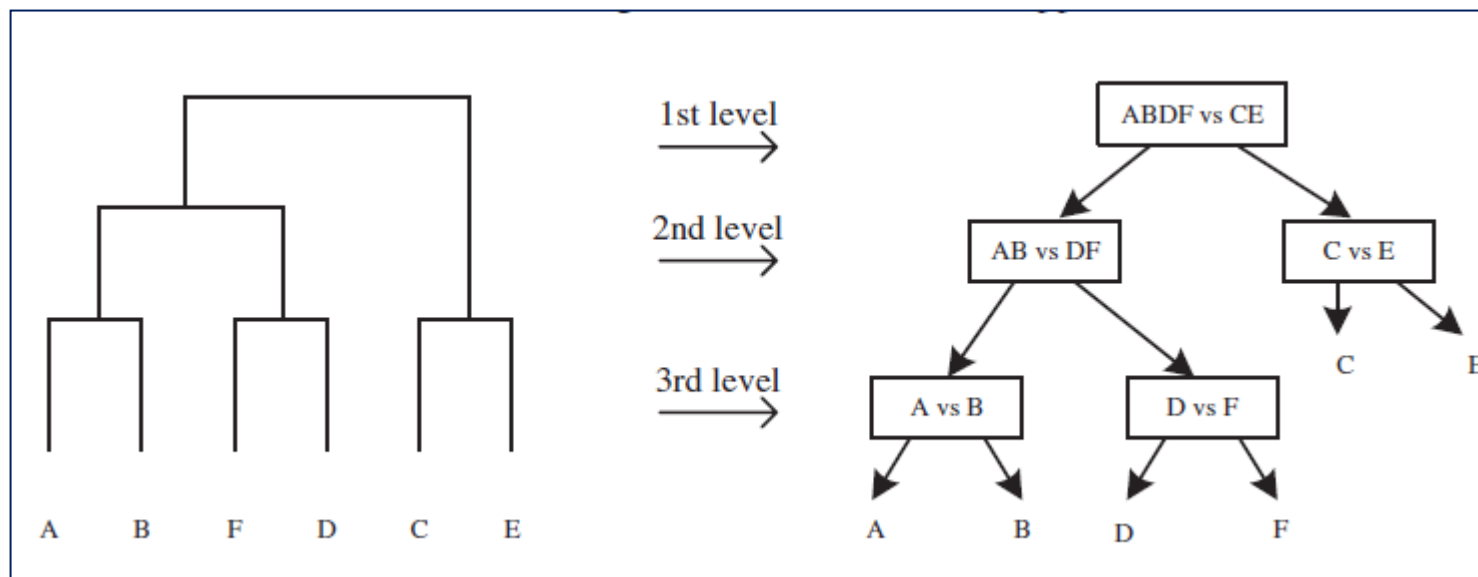
DAGSVM 通过构建一个有向无环图（DAG）来表示多个分类器，每个节点代表一个二分类器，节点之间的边表示类别的决策边界。

Algorithm:

1. Create a list of class labels $L=(1,2,\dots,M)$ (arbitrary order)
2. Evaluates the sample with the binary SVM that corresponds to the **first** and **last** element in list L , the loser class index is eliminated from the list. (失败的类别逐渐被过滤掉)
3. After $M-1$ evaluations, the last label is the answer.

15.9.3 Half-vs-Half

可以利用K-means或层级聚类方法来对类别进行分组



- ✓ H. Lei, V. Govindaraju, Half-Against-Half Multi-Class Support Vector Machines, *MCS 2005*.
- ✓ V. Vural, J.G. Dy, A Hierarchical Method for Multi-Class Support Vector Machines, *ICML 2004*.

15.9.3 Half-vs-Half

- How to divide the data into two subsets hierarchically?
 - ✓ Two subsets have the largest margin.
 - ✓ Two subsets have minimum number of SVs
 - ✓ k-means division
 - ✓ Spherical shells
 - ✓ Balanced subsets
 - ✓ Hierarchy clustering
- Testing speed: worst case: **M-1** evaluations
 - ✓ 1-v-1: $M(M-1)/2$
 - ✓ 1-v-a: M
 - ✓ DAGSVM: $M-1$

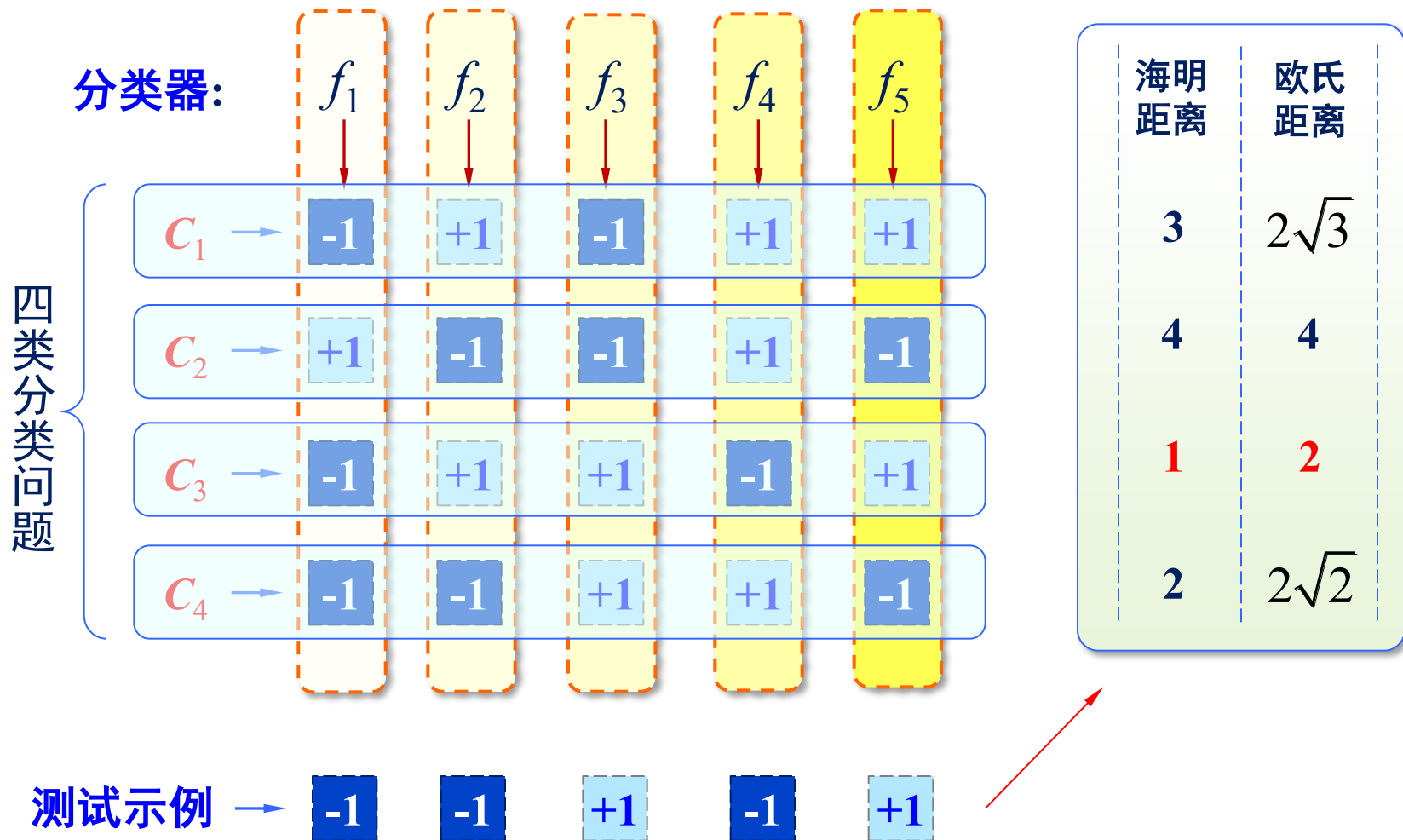
常用的二分组方法

15.9.4 ECOC

- 类别划分通过编码矩阵（coding matrix）进行来指定。编码矩阵有多种形式。常见的有二元码、三元码。
 - 二元码：将每个类别，要么指定为正类，要么指定为负类；
 - 三元码：对于一个类别，既可以充当正类，也可以充当负类，还可以充当“停用类”。

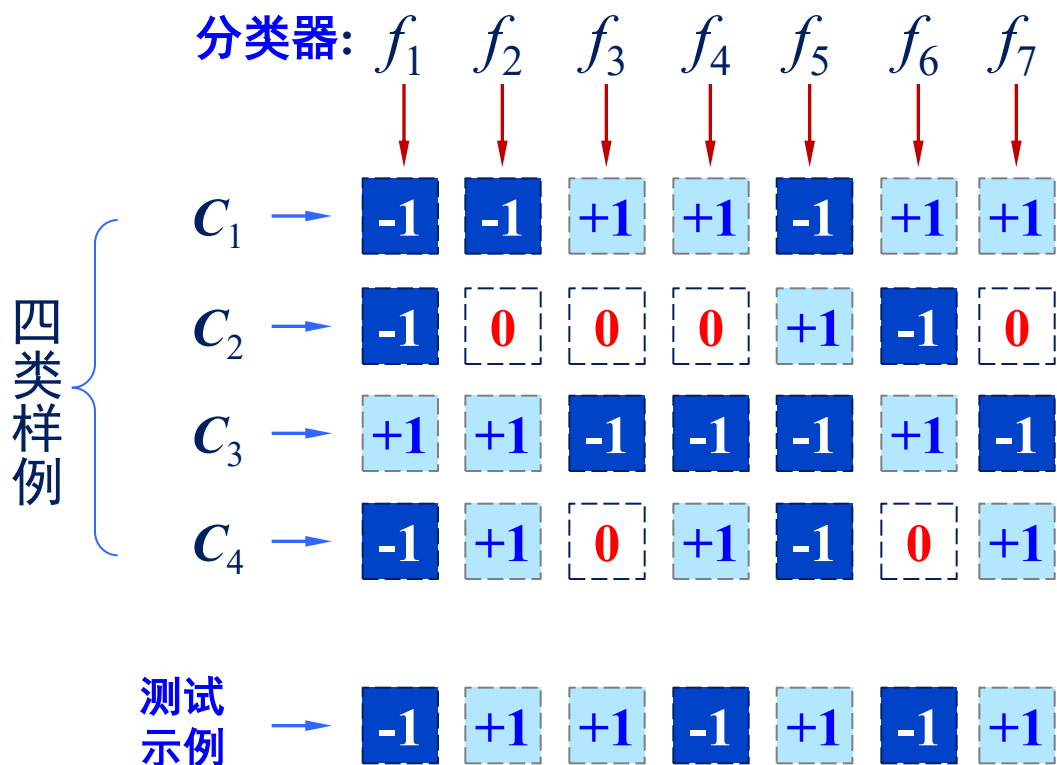
15.9.4 ECOC

- ECOC二元码示意图（以四类分类问题为例）



15.9.4 ECOC

- ECOC三元码示意图 (0:表示不考虑此类):



海明距离	欧氏距离
4	4
4	2
5	$2\sqrt{5}$
3	$\sqrt{10}$

15.9.4 ECOC

• ECOC为什么会纠错

- 在测试阶段，ECOC编码对分类器的错误有一定的容忍和修正能力。
 - 假设在上述的二元码示例中，正确的预测编码应该为 $(-1, +1, +1, -1, +1)$ ，即属于 C_3 类。但多分类器的实际输出为 $(-1, -1, +1, -1, +1)$ 。最后仍被分 C_3 类。
- 对同一个学习任务，ECOC编码越长，纠错能力就越强。
- 对同等长度的编码，理论上讲，任意两个类别之间的编码越远，则纠错能力越强。
- 在编码长度较小时，可以据此设计最优编码。但对于长编码，最优编码设计是一个NP难问题。
- 但并不是编码的理论性能越好，分类性能就越好，因为机器学习问题涉及很多因素。
- 编码越长，则分类器越多，训练时间越长。

15.9.5 其它方法（略）

✓ **LatticeSVM**: 通过引入“格子算法”，解决了传统SVM在多类分类问题上的不足。

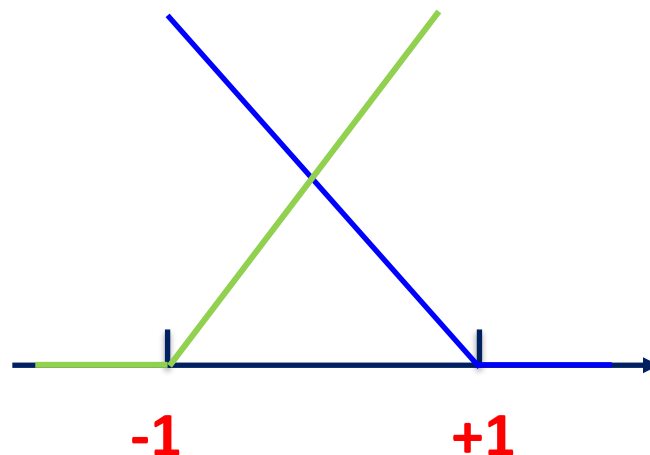
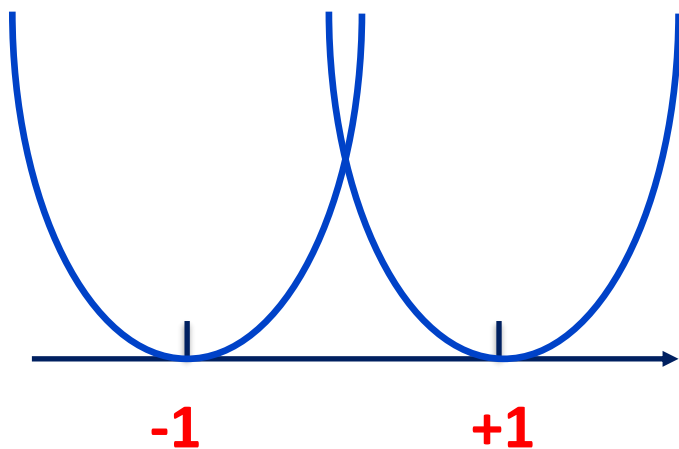
- LatticeSVM通过静态候选技术和格子结构，有效地处理了多类问题，并在存储空间和运算速度上具有明显优势
- Z. Liu, L. Jin, LatticeSVM—A New Method for Multi-Class Support Vector Machines, *IJCNN 2008*.
-

15.10 支持向量机与判别最小二乘法

A more natural way to solve multi-class problems is to construct a decision function by considering all classes at once. Result in a much larger optimization problem in one step.

15.10.1 基础知识回顾

Least Square Loss vs Hinge Loss:



15.10.1 基础知识回顾

Linear Regression:

- Data set $\{\mathbf{x}_i\}_{i=1}^n \subset R^m$
- Destination set $\{\mathbf{y}_i\}_{i=1}^n \subset R^c$
- Linear regression can be defined as:

$$\min_{\mathbf{W}, \mathbf{b}} \sum_{i=1}^n \|\mathbf{W}^T \mathbf{x}_i + \mathbf{b} - \mathbf{y}_i\|_2^2 + \lambda \|\mathbf{W}\|_F^2, \quad \mathbf{W} \in R^{m \times c}, \quad \mathbf{b} \in R^c$$

- $\|\cdot\|_2$: Vector L_2 norm
- $\|\cdot\|_F$: Matrix Frobenius norm
- The simplest (multi-class) classifier !

15.10.1 基础知识回顾

Different Loss Functions:

- Data: $\{\mathbf{x}_i\}_{i=1}^n \subset R^m$
- Classifier: $y = \text{sign}(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b)$
- Margin variable: $z = t y, \quad t \in \{+1, -1\}$
 - Large $z \rightarrow$ small loss
- Loss Functions:
 - 0-1 loss: $l_{\text{mer}}(z_i) = \|(-z_i)_+\|_0$
 - Hinge loss: $l_{\text{hinge}} = \{(1 - z_i)_+\}^p \quad (p=1 \text{ or } 2)$
 - Logistic regression: $l_{\text{log}} = \log_2[1 + \exp(-z_i)]$
 - Least squares: $l_{\text{ls}} = (1 - z_i)^2$
 - Adaboost: $l_{\text{exp}} = \exp(-z_i).$

15.10.2 Discriminative LSR

- Data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n \quad y_i \in \{1, 2, \dots, c\}$
 - Class $\mathbf{f}_j = [0, \dots, 0, 1, 0, \dots, 0]^T \in \mathbb{R}^c$
-

- Training sample fitting

$$\mathbf{f}_{y_i} \approx \mathbf{W}^T \mathbf{x}_i + \mathbf{t}, \quad i = 1, 2, \dots, n$$

- Let $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n]^T \in \mathbb{R}^{n \times m}$
 $\mathbf{Y} = [\mathbf{f}_{y_1}, \mathbf{f}_{y_2}, \dots, \mathbf{f}_{y_n}]^T \in \mathbb{R}^{n \times c}$
-

- Training sample fitting

$$\mathbf{XW} + \mathbf{e}_n \mathbf{t}^T \approx \mathbf{Y}$$

Shiming Xiang et al. Discriminative Least Squares Regression for Multiclass Classification and Feature Selection, IEEE T-NNLS, 2012.

15.10.2 Discriminative LSR

$$\mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T \approx \mathbf{Y}$$

- Each column of $\mathbf{Y} \rightarrow$ binary regression with +1/0
- How to enlarge the margin?
- Define a new matrix $\mathbf{B} \in \mathbb{R}^{n \times c}$

$$B_{ij} = \begin{cases} +1, & \text{if } y_i = j \\ -1, & \text{otherwise.} \end{cases}$$

- Each element in $\mathbf{B}$ corresponds to a dragging direction
- Target re-definition

$$\mathbf{M} \in \mathbb{R}^{n \times c}$$

$$\mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - (\mathbf{Y} + \mathbf{B} \odot \mathbf{M}) \approx \mathbf{0}$$

15.10.2 Discriminative LSR

- Optimize three parts of parameters

$$\min_{\mathbf{W}, \mathbf{t}, \mathbf{M}} \left\| \mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - (\mathbf{Y} + \mathbf{B} \odot \mathbf{M}) \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2$$
$$s.t. \quad \mathbf{M} \geq \mathbf{0}$$

- OPT1:
 - Fix $\mathbf{M}$ and solve $\mathbf{W}, \mathbf{t} \rightarrow$ unconstrained convex QP $\rightarrow$ 解析解
- OPT2:
 - Fix $\mathbf{W}, \mathbf{t}$ and solve $\mathbf{M} \rightarrow$ very simple elementwise solution

15.10.2 Discriminative LSR

OPT1: Fix $\mathbf{M}$

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{M}} \quad & \left\| \mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - (\mathbf{Y} + \mathbf{B} \odot \mathbf{M}) \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & \mathbf{M} \geq \mathbf{0} \end{aligned}$$

- Fix $\mathbf{M}$, and let $\mathbf{R} = \mathbf{Y} + \mathbf{B} \odot \mathbf{M} \in \mathbb{R}^{n \times c}$.
- We have:

$$\begin{aligned} \mathbf{W} &= (\mathbf{X}^T \mathbf{H} \mathbf{X} + \lambda \mathbf{I}_m)^{-1} \mathbf{X}^T \mathbf{H} \mathbf{R} \\ \mathbf{t} &= \frac{(\mathbf{R}^T \mathbf{e}_n - \mathbf{W}^T \mathbf{X}^T \mathbf{e}_n)}{n} \end{aligned}$$

15.10.2 Discriminative LSR

OPT1: Fix $\mathbf{M}$

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{M}} \quad & \left\| \mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - (\mathbf{Y} + \mathbf{B} \odot \mathbf{M}) \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & \mathbf{M} \geq \mathbf{0} \end{aligned}$$

$$\begin{aligned} \frac{\partial g(\mathbf{W}, \mathbf{t})}{\partial \mathbf{t}} = \mathbf{0} & \Rightarrow \mathbf{W}^T \mathbf{X}^T \mathbf{e}_n + \mathbf{t} \mathbf{e}_n^T \mathbf{e}_n - \mathbf{R}^T \mathbf{e}_n = \mathbf{0} \\ & \Rightarrow \mathbf{t} = \frac{(\mathbf{R}^T \mathbf{e}_n - \mathbf{W}^T \mathbf{X}^T \mathbf{e}_n)}{n}. \end{aligned}$$

$$\begin{aligned} \frac{\partial g(\mathbf{W}, \mathbf{t})}{\partial \mathbf{W}} = \mathbf{0} \\ \Rightarrow \mathbf{X}^T \left(\mathbf{XW} + \frac{1}{n} \mathbf{e}_n \mathbf{e}_n^T \mathbf{R} - \frac{1}{n} \mathbf{e}_n \mathbf{e}_n^T \mathbf{XW} - \mathbf{R} \right) + \lambda \mathbf{W} = \mathbf{0} \\ \Rightarrow \mathbf{X}^T \left(\mathbf{I}_n - \frac{1}{n} \mathbf{e}_n \mathbf{e}_n^T \right) \mathbf{XW} - \mathbf{X}^T \left(\mathbf{I}_n - \frac{1}{n} \mathbf{e}_n \mathbf{e}_n^T \right) \mathbf{R} + \lambda \mathbf{W} = \mathbf{0} \\ \Rightarrow \mathbf{W} = (\mathbf{X}^T \mathbf{H} \mathbf{X} + \lambda \mathbf{I}_m)^{-1} \mathbf{X}^T \mathbf{H} \mathbf{R}. \end{aligned}$$

15.10.2 Discriminative LSR

OPT2: Fix $\mathbf{W}$ and $\mathbf{t}$

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{M}} \quad & \left\| \mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - (\mathbf{Y} + \mathbf{B} \odot \mathbf{M}) \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & \mathbf{M} \geq \mathbf{0} \end{aligned}$$

$$\mathbf{P} = \mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - \mathbf{Y}$$

$$\min_{\mathbf{M}} \quad \left\| \mathbf{P} - \mathbf{B} \odot \mathbf{M} \right\|_F^2, \quad \text{s.t.} \quad \mathbf{M} \geq \mathbf{0}.$$

$$\min_{M_{ij}} \quad (P_{ij} - B_{ij} M_{ij})^2, \quad \text{s.t.} \quad M_{ij} \geq 0$$

$$M_{ij} = \max(B_{ij} P_{ij}, 0).$$

$$\mathbf{M} = \max(\mathbf{B} \odot \mathbf{P}, \mathbf{0}).$$

15.10.2 Discriminative LSR

Algorithm of DLSR

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{M}} \quad & \|\mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - \mathbf{Y} - \mathbf{B} \odot \mathbf{M}\|_F^2 + \lambda \|\mathbf{W}\|_F^2 \\ \text{s.t.} \quad & \mathbf{M} \geq \mathbf{0} \end{aligned}$$

Algorithm 1 *DLSR*

Input: n data points $\{\mathbf{x}_i\}_{i=1}^n$ in $\mathbb{R}^m$, and their corresponding class labels $\{y_i\}_{i=1}^n \subset \{1, 2, \dots, c\}$; parameter λ in (7); and maximum number of iterations T .

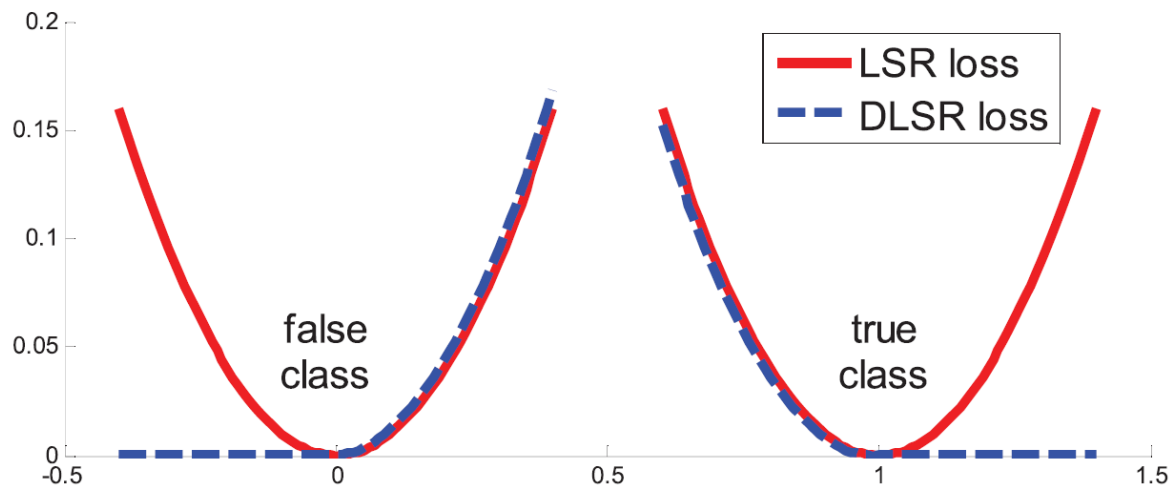
- 1: Allocate $\mathbf{M}$, $\mathbf{W}$, $\mathbf{W}_0$, $\mathbf{t}$, and $\mathbf{t}_0$.
 - 2: $\mathbf{M} = \mathbf{0}$, $\mathbf{W}_0 = \mathbf{0}$, and $\mathbf{t}_0 = \mathbf{0}$.
 - 3: Construct $\mathbf{X}$ and $\mathbf{Y}$ in (4), and $\mathbf{B}$ according to (5).
 - 4: Let $\mathbf{U} = (\mathbf{X}^T \mathbf{H} \mathbf{X} + \lambda \mathbf{I}_m)^{-1} \mathbf{X}^T \mathbf{H}$.
 - 5: Let $k = 1$.
 - 6: **while** $k < T$ **do**
 - 7: $\mathbf{R} = \mathbf{Y} + \mathbf{B} \odot \mathbf{M}$
 - 8: $\mathbf{W} = \mathbf{UR}$, $\mathbf{t} = \frac{1}{n} \mathbf{R}^T \mathbf{e}_n - \frac{1}{n} \mathbf{W}^T \mathbf{X}^T \mathbf{e}_n$.
 - 9: $\mathbf{P} = \mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - \mathbf{Y}$.
 - 10: $\mathbf{M} = \max(\mathbf{B} \odot \mathbf{P}, \mathbf{0})$.
 - 11: **if** $(\|\mathbf{W} - \mathbf{W}_0\|_F^2 + \|\mathbf{t} - \mathbf{t}_0\|_2^2) < 10^{-4}$, **then**
 - 12: Stop.
 - 13: **end if**
 - 14: $\mathbf{W}_0 = \mathbf{W}$, $\mathbf{t}_0 = \mathbf{t}$, $k = k + 1$.
 - 15: **end while**
 - 16: Output $\mathbf{W}$ and $\mathbf{t}$.
-

15.10.2 Discriminative LSR

Re-think DLSR

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{M}} \quad & \left\| \mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - (\mathbf{Y} + \mathbf{B} \odot \mathbf{M}) \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & \mathbf{M} \geq \mathbf{0} \end{aligned}$$

- Another formalization of **one-vs-all SVM** with **squared hinge loss** !



15.10.3 Retargeted LSR

- LSR: use **exact 0-1** as target
- DLSR: use **relaxed 0-1** as target
- ReLSR: use **any (learned) number** as target
 - Totally learn the regression targets from data
 - With flexible large margin constraints

y	regression result	margin ≥ 1	LSR	DLSR	ReLSR
[1, 0, 0]	[1.5, 0, 0]	Yes	loss=0.25	loss=0.00 target=[1.5, 0, 0]	loss=0.00 target=[1.5, 0, 0]
[1, 0, 0]	[1, -0.5, -0.5]	Yes	loss=0.50	loss=0.00 target=[1, -0.5, -0.5]	loss=0.00 target=[1, -0.5, -0.5]
[0, 1, 0]	[0.5, 1.5, 0.5]	Yes	loss=0.75	loss=0.50 target=[0, 1.5, 0]	loss=0.00 target=[0.5, 1.5, 0.5]
[0, 1, 0]	[-0.5, 1.5, 0.5]	Yes	loss=0.75	loss=0.25 target=[-0.5, 1.5, 0]	loss=0.00 target=[-0.5, 1.5, 0.5]
[0, 0, 1]	[0.2, 0.2, 0.8]	No	loss=0.12	loss=0.12 target=[0, 0, 1]	loss=0.11 target=[0.0667, 0.0667, 1.0667]
[0, 0, 1]	[-0.2, 0.2, 0.6]	No	loss=0.24	loss=0.20 target=[-0.2, 0, 1]	loss=0.18 target=[-0.2, -0.1, 0.9]

Xu-Yao Zhang, Lingfeng Wang, Shiming Xiang, Cheng-Lin Liu. Retargeted Least Squares Regression Algorithm. IEEE Transactions on Neural Network and Learning Systems (T-NNLS), 2015.

15.10.3 Retargeted LSR

- Target Matrix $\mathbf{T} \in \mathbb{R}^{n \times c}$

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{T}} \quad & \left\| \mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - \mathbf{T} \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & T_{i, y_i} - \max_{j \neq y_i} T_{i, j} \geq 1, \quad i = 1, 2, \dots, n \end{aligned}$$

- The constraint flexibility: $\text{LSR} < \text{DLSR} < \text{ReLSR}$
- LSR/DLSR $\rightarrow$ decomposed into c independent sub-problems (one-against-rest)
- Because of $\mathbf{T}$, ReLSR is a single and compact machine for multiclass classification

15.10.3 Retargeted LSR

OPT1: Regression

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{T}} \quad & \left\| \mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - \mathbf{T} \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & T_{i, y_i} - \max_{j \neq y_i} T_{i, j} \geq 1, \quad i = 1, 2, \dots, n \end{aligned}$$

Regression: $\min_{\mathbf{W}, \mathbf{t}} \left\| \mathbf{XW} + \mathbf{e}_n \mathbf{t}^T - \mathbf{T} \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2$

$$\mathbf{W} = \left(\mathbf{X}^T \mathbf{H} \mathbf{X} + \lambda \mathbf{I}_d \right)^{-1} \mathbf{X}^T \mathbf{H} \mathbf{T}, \quad \mathbf{t} = \frac{1}{n} \left(\mathbf{T} - \mathbf{XW} \right)^T \mathbf{e}_n$$

15.10.3 Retargeted LSR

OPT2: Retargeting

$$\begin{aligned} \min_{\mathbf{W}, \mathbf{t}, \mathbf{T}} \quad & \left\| \mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - \mathbf{T} \right\|_F^2 + \lambda \left\| \mathbf{W} \right\|_F^2 \\ \text{s.t.} \quad & T_{i, y_i} - \max_{j \neq y_i} T_{i, j} \geq 1, \quad i = 1, 2, \dots, n \end{aligned}$$

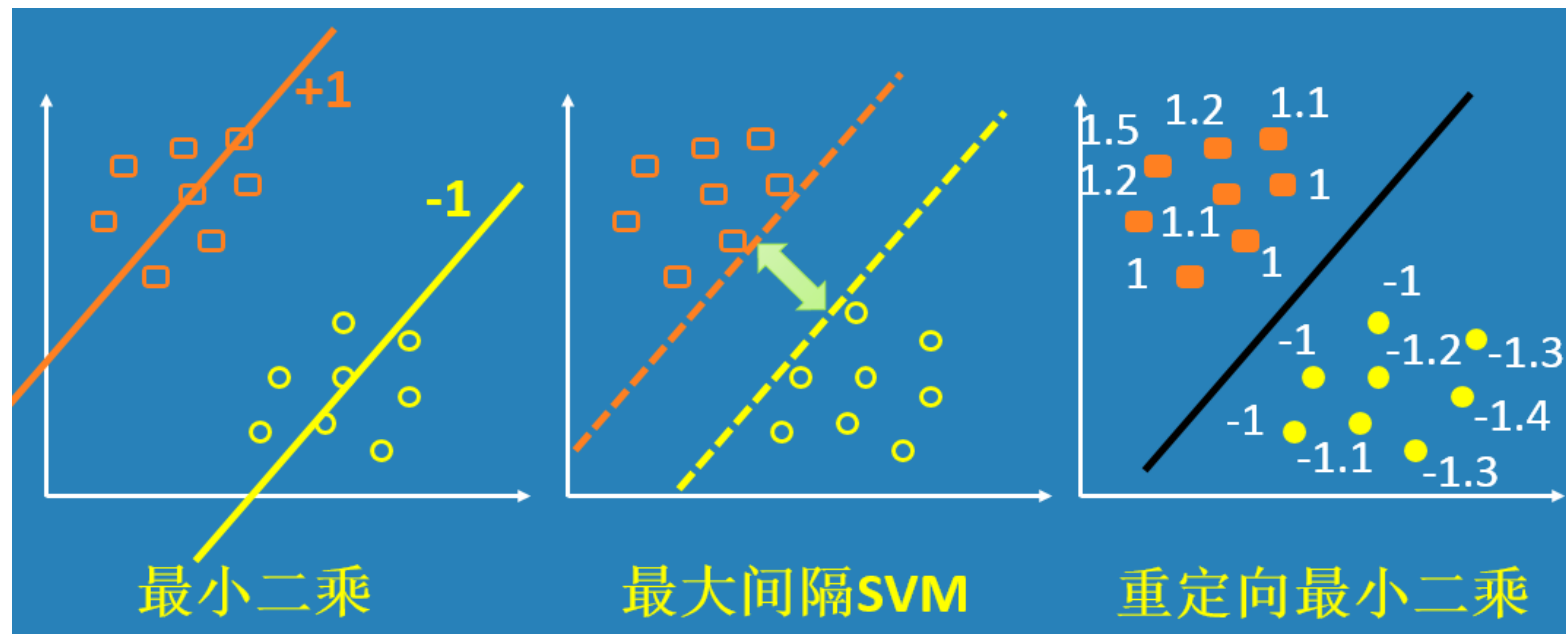
Retargeting

$$\begin{aligned} \min_{\mathbf{T}} \quad & \left\| \mathbf{X}\mathbf{W} + \mathbf{e}_n \mathbf{t}^T - \mathbf{T} \right\|_F^2 = \left\| \mathbf{R} - \mathbf{T} \right\|_F^2 \\ \text{s.t.} \quad & T_{i, y_i} - \max_{j \neq y_i} T_{i, j} \geq 1, \quad i = 1, 2, \dots, n \end{aligned}$$

$$\min_{\mathbf{s}} \left\| \mathbf{r} - \mathbf{s} \right\|_2^2 = \sum_{i=1}^c (r_i - s_i)^2, \quad \text{s.t.} \quad s_k - \max_{i \neq k} s_i \geq 1$$

15.10.3 Retargeted LSR

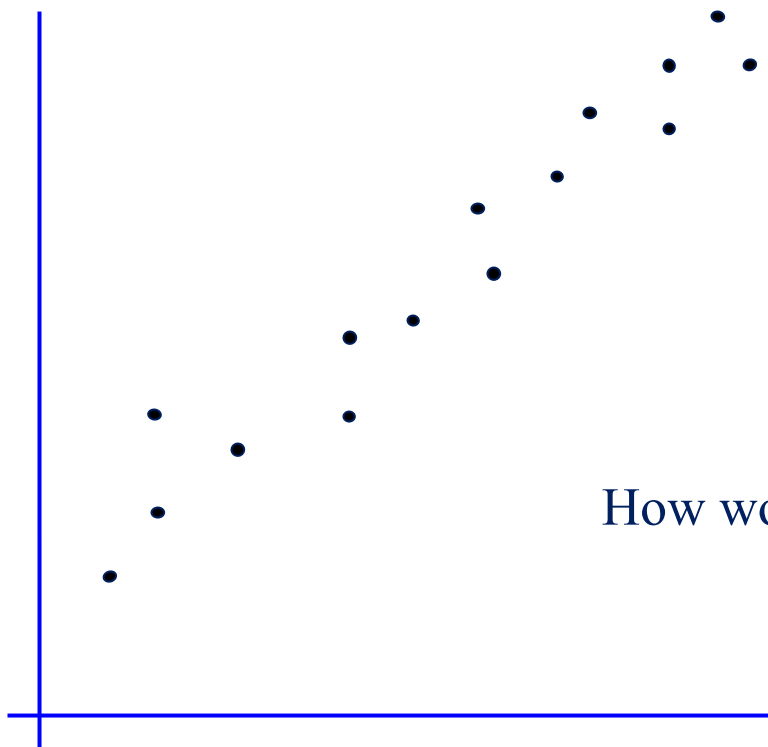
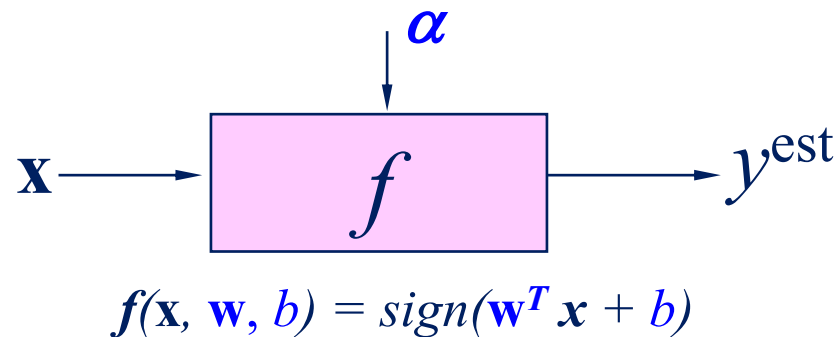
From Regression to Classification



Xu-Yao Zhang, Lingfeng Wang, Shiming Xiang, Cheng-Lin Liu. Retargeted Least Squares Regression Algorithm. IEEE Transactions on Neural Network and Learning Systems (TNNLS), 2015.

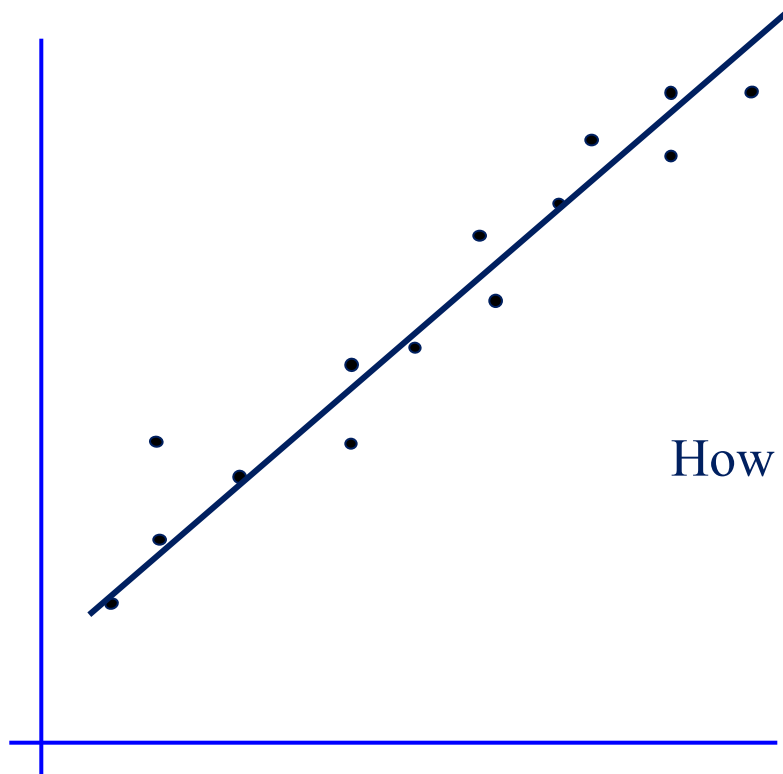
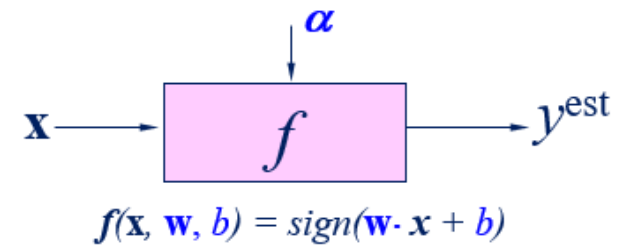
15.11 Support Vector Regression

15.11.1 Linear Regression



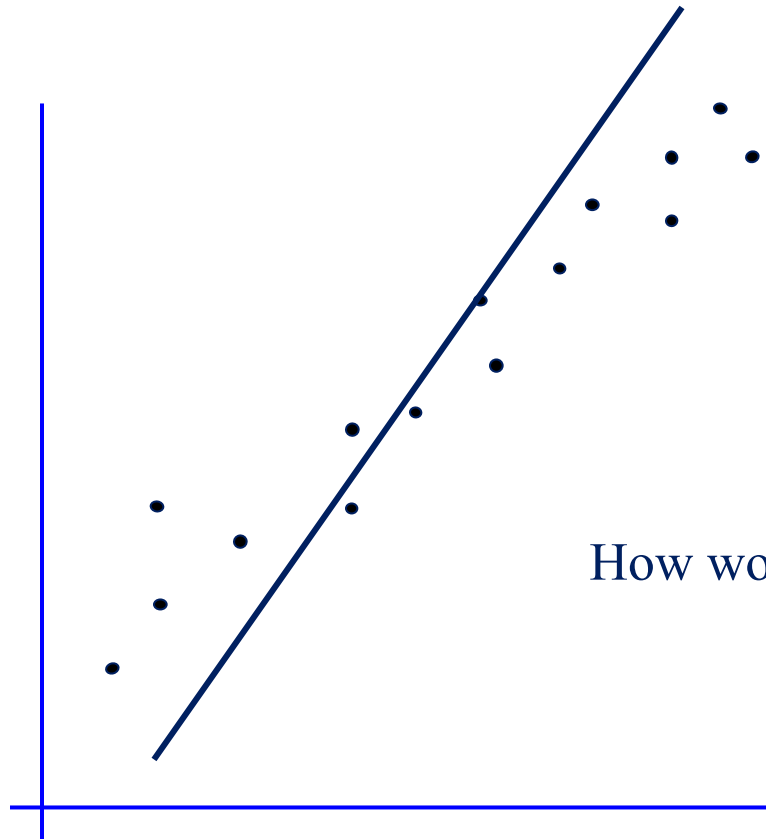
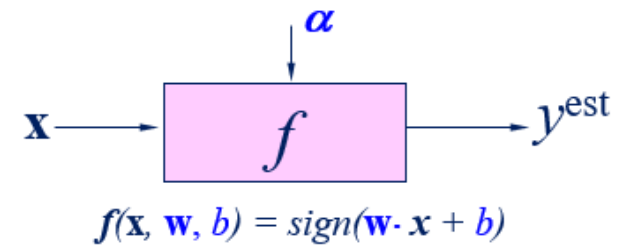
How would you fit this data?

15.11.1 Linear Regression



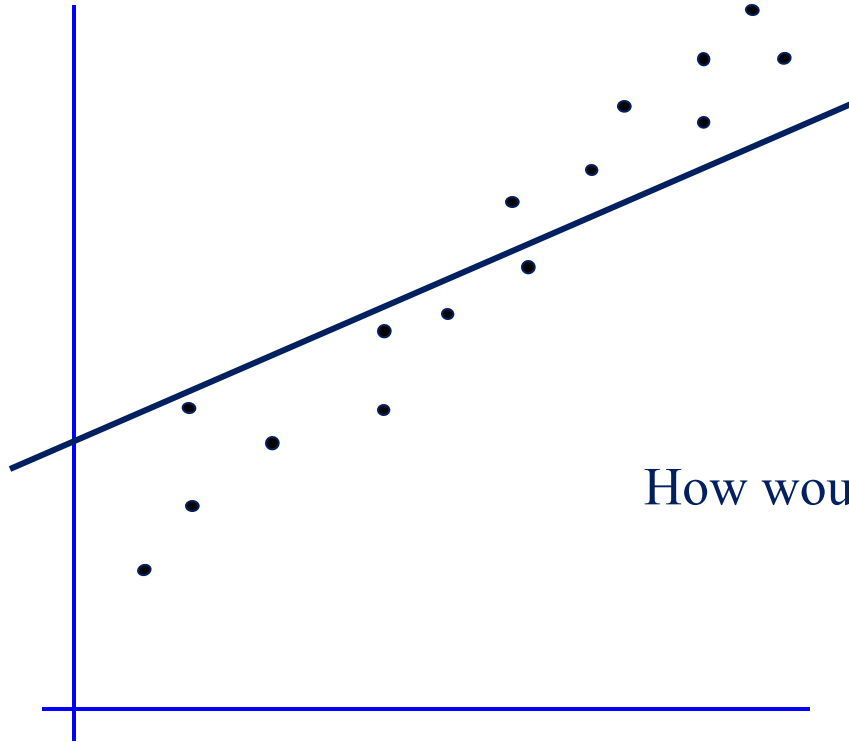
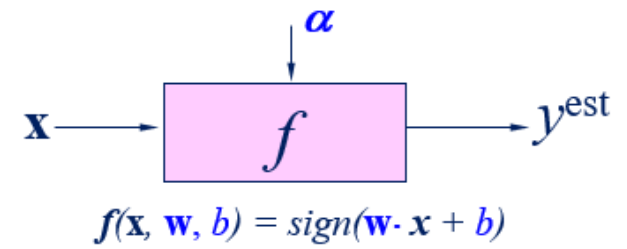
How would you fit this data?

15.11.1 Linear Regression



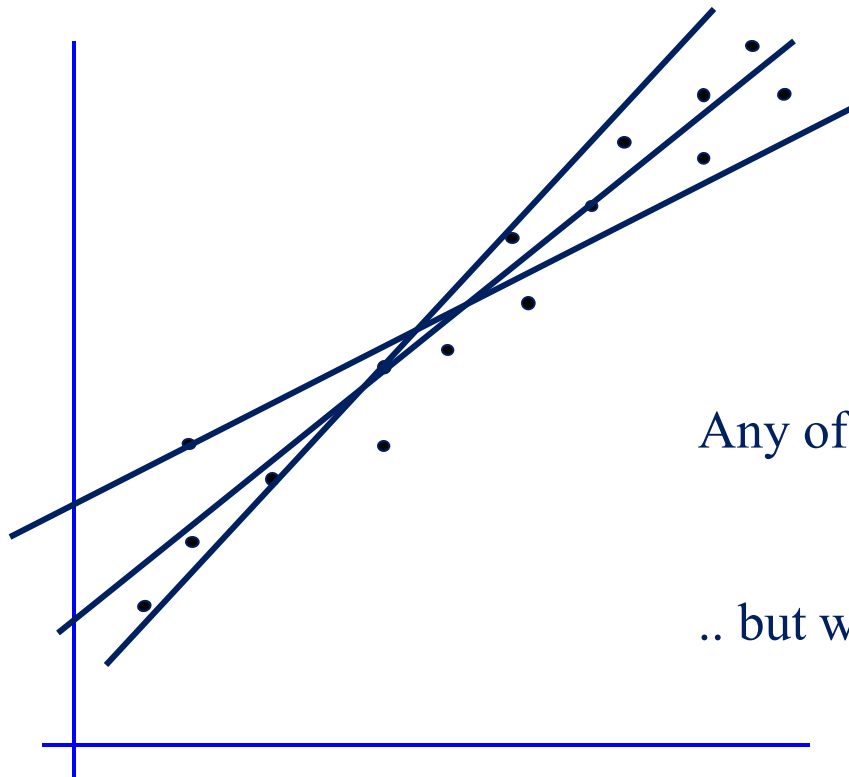
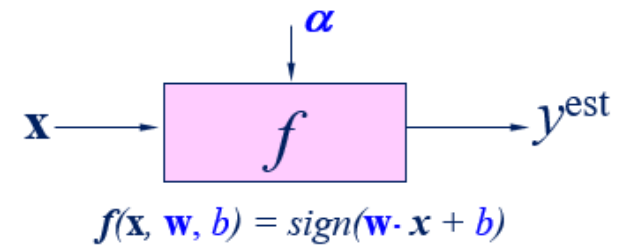
How would you fit this data?

15.11.1 Linear Regression



How would you fit this data?

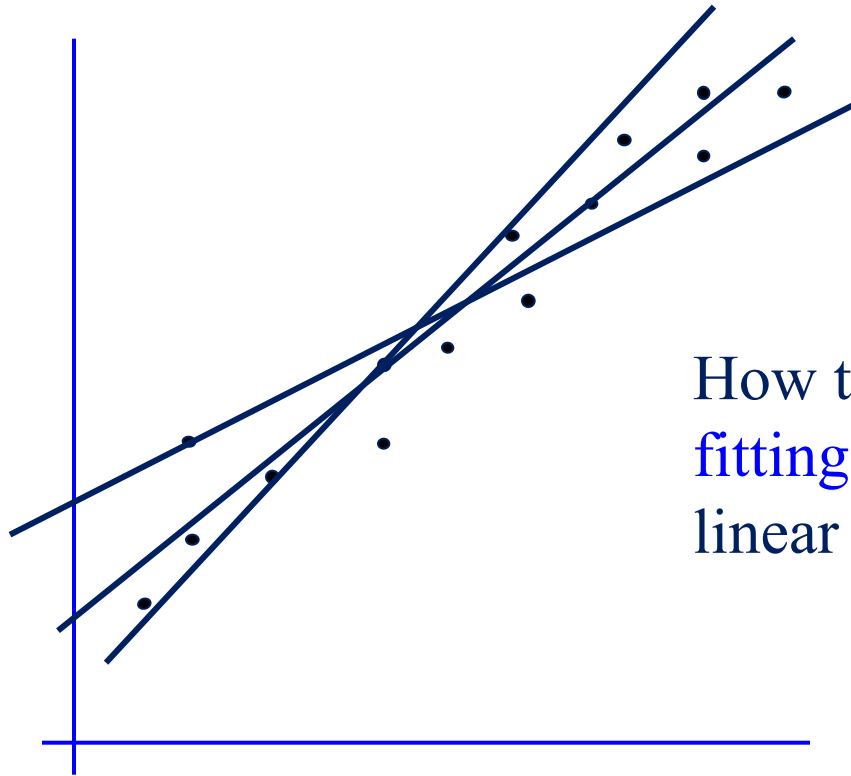
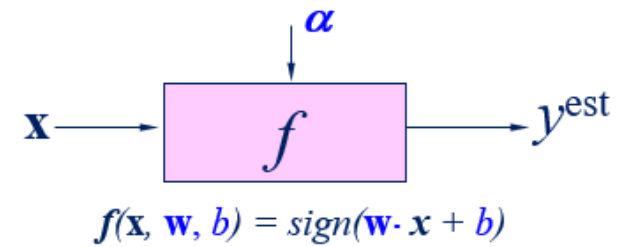
15.11.1 Linear Regression



Any of these would be fine..

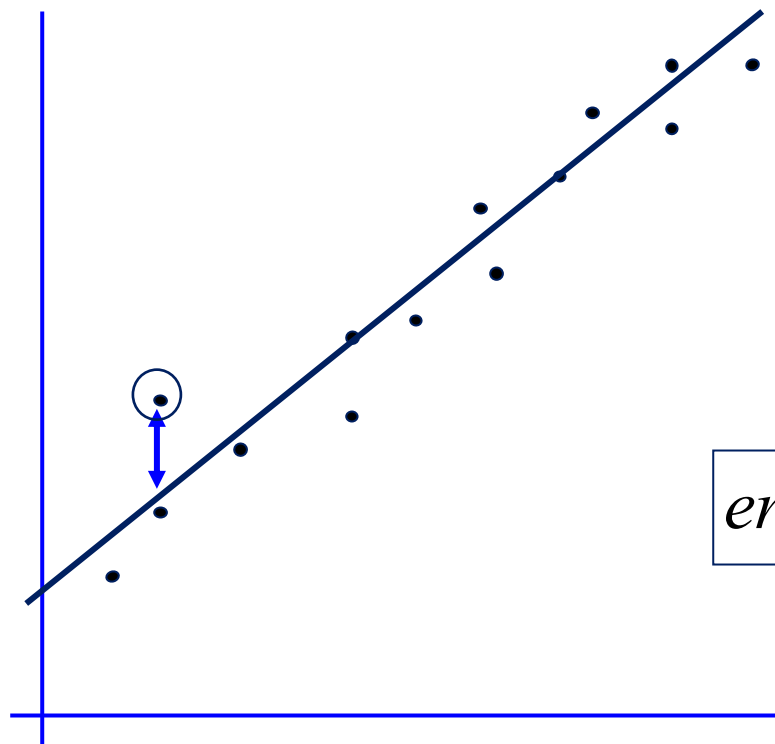
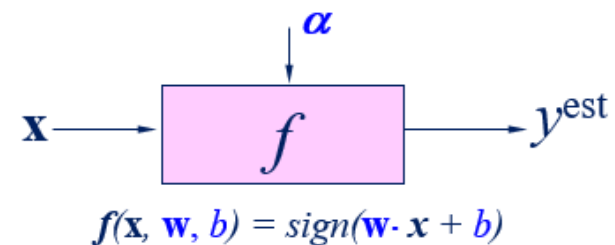
.. but which is best?

15.11.1 Linear Regression



How to define the
fitting error of a
linear regression ?

15.11.1 Linear Regression



How to define the **fitting error** of a linear regression ?

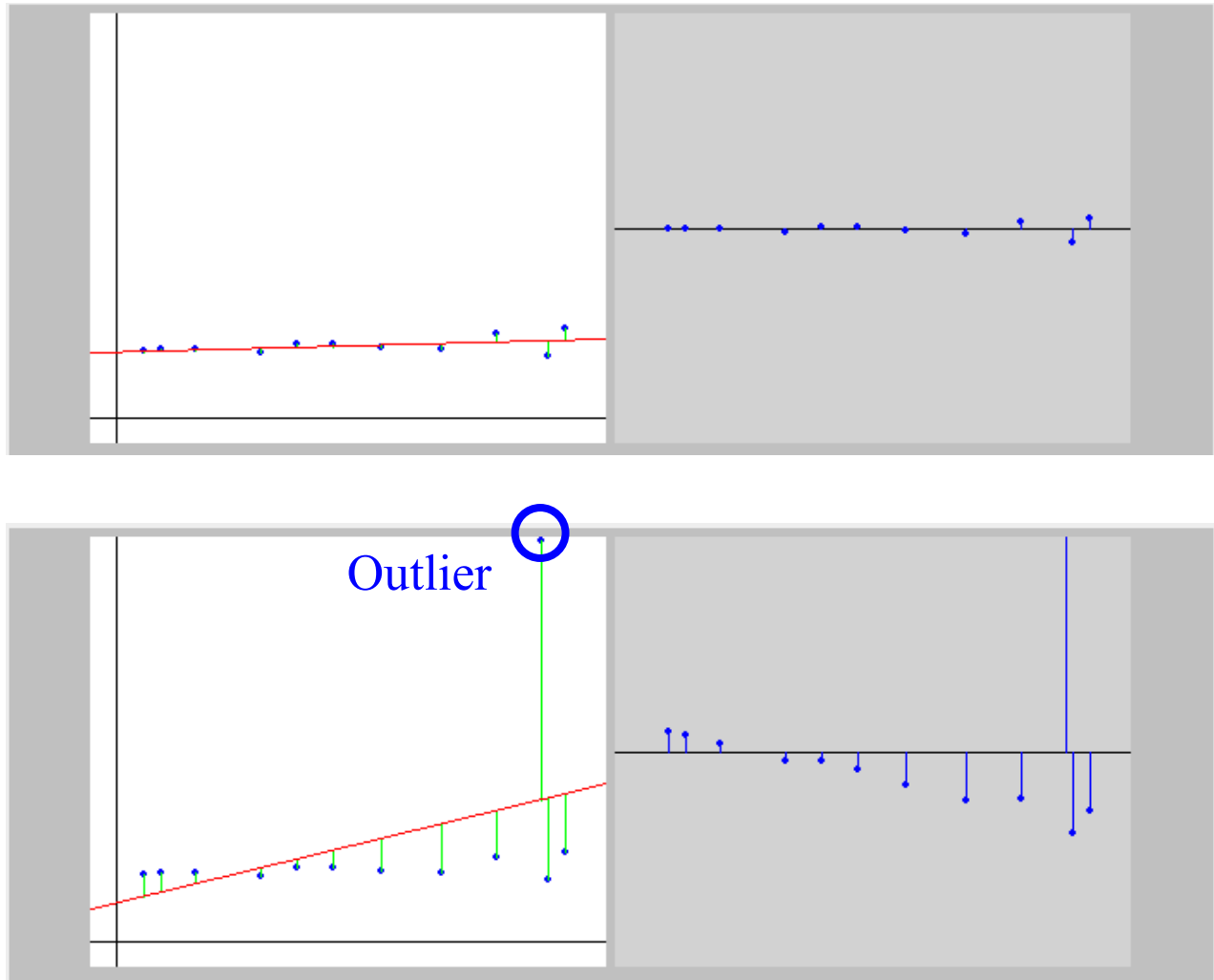


$$err_i = (\mathbf{w} \cdot \mathbf{x}_i - b - y_i)^2$$

Squared-Loss

15.11.1 Linear Regression

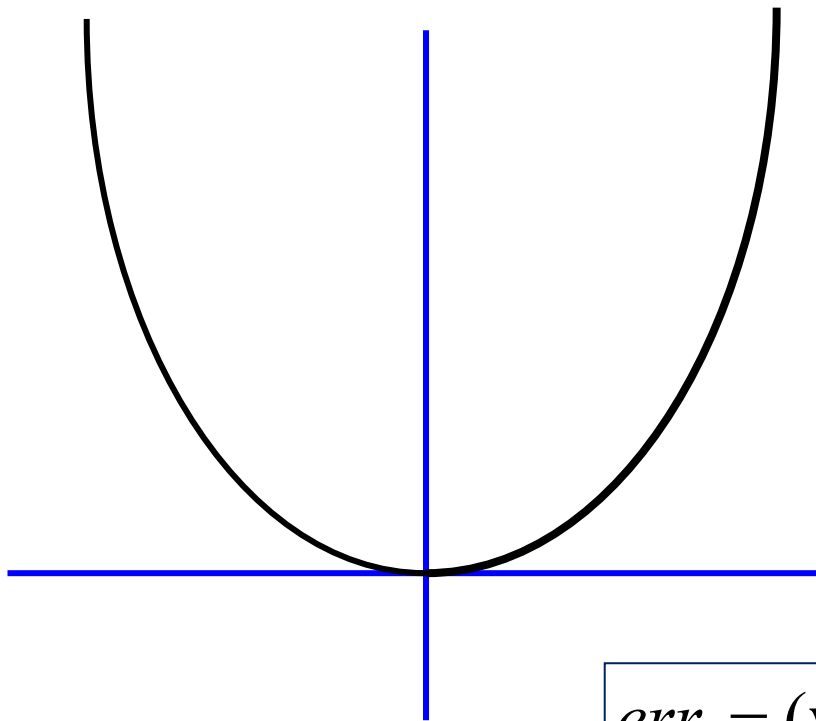
Sensitive to Outliers:



15.11.1 Linear Regression

Why?

- Squared-Loss Function: **Fitting Error Grows Quadratically**

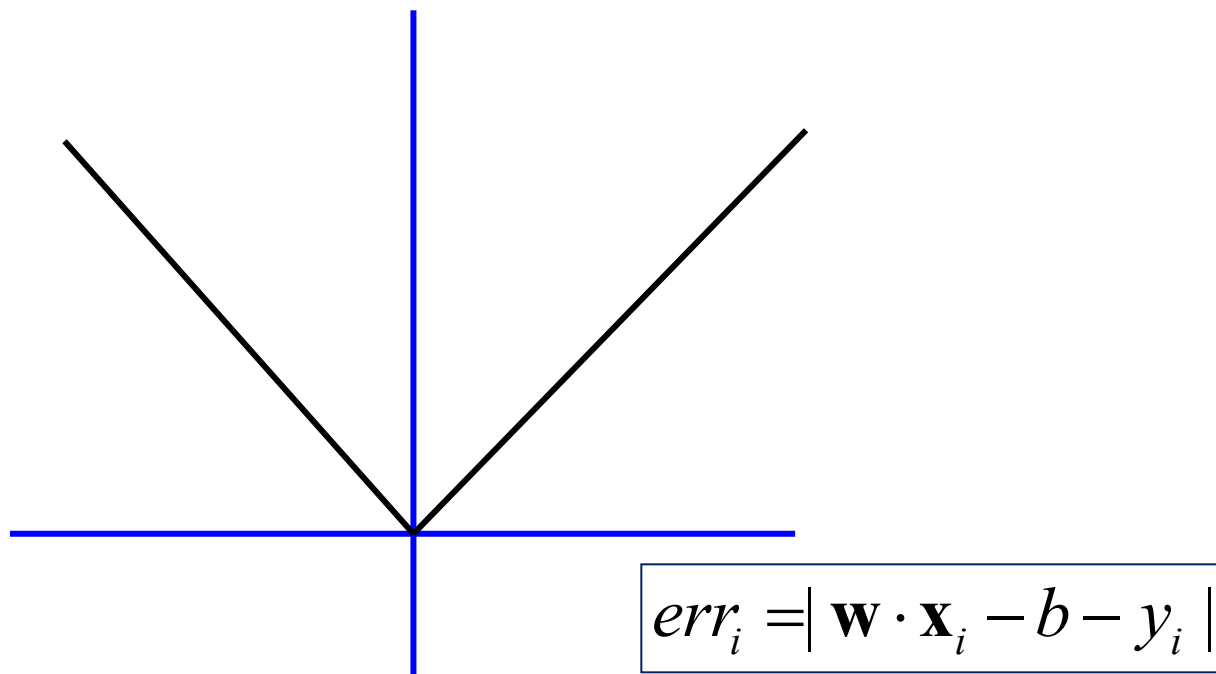


$$err_i = (\mathbf{w} \cdot \mathbf{x}_i - b - y_i)^2$$

15.11.1 Linear Regression

How about Linear-Loss ?

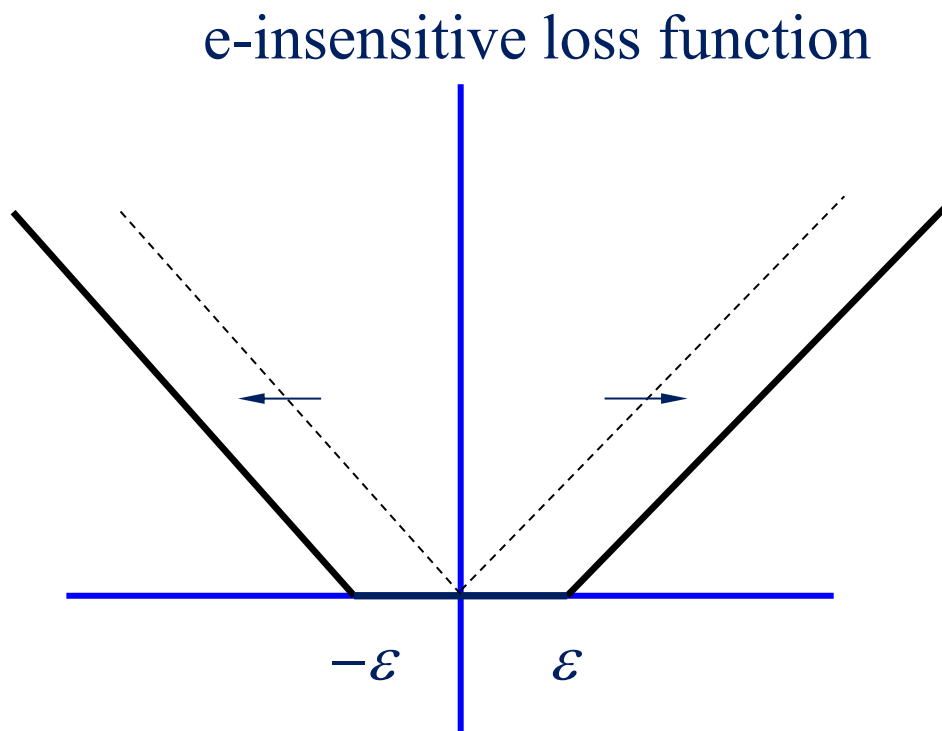
- Linear-Loss Function: Fitting Error Grows Linearly



15.11.2 支持向量机回归

Actually

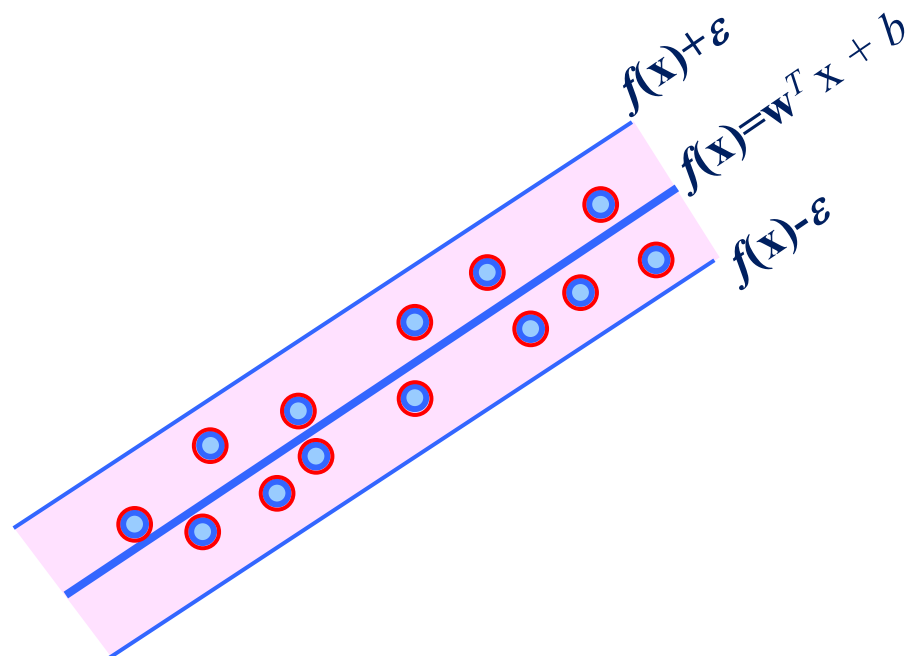
- Support vector regression (SVR) uses the Loss Function below



回归差异比较小时，比如小于 ϵ 时，直接令损失等于0

15.11.2 支持向量机回归

- 基本思想：与分类任务不同，SVR 不追求“完全正确预测”，而是允许预测值与真实值存在一定范围内的误差（ ε -不敏感损失函数），通过最小化模型复杂度实现强泛化能力。假设 $f(\mathbf{x})$ 与 y 之间可以有 ε 容忍偏差，在这个偏差内，可接收。



如果所有的样本点都在一个宽度为 2ε 的管道内，我们得到了一个很好的回归！

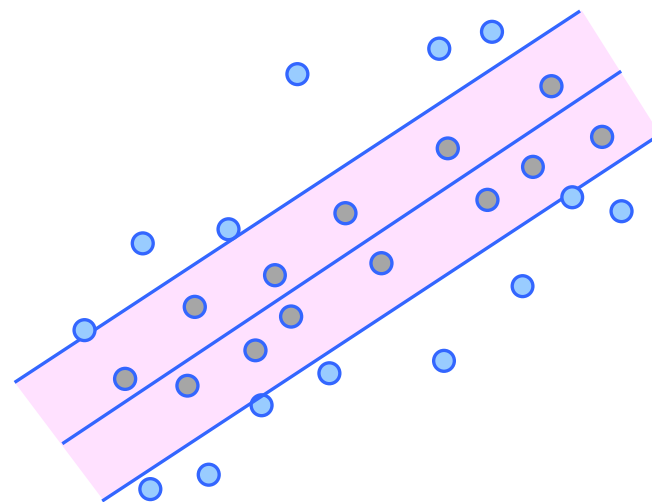
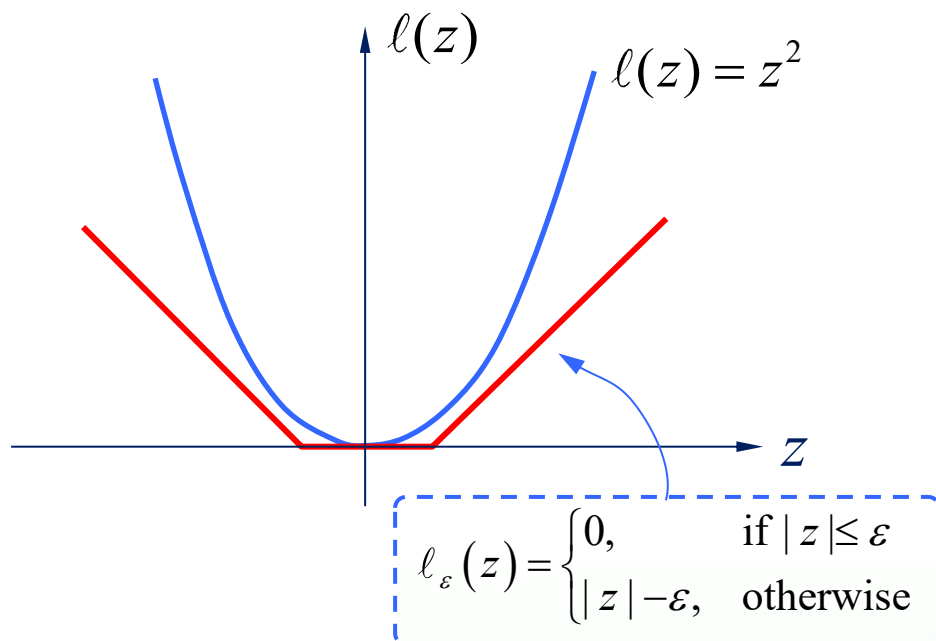
15.11.2 支持向量机回归

• 学习模型

$$\min_{\mathbf{w}, \mathbf{b}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \ell_{\varepsilon}(\underbrace{f(\mathbf{x}_i) - y_i}_z),$$

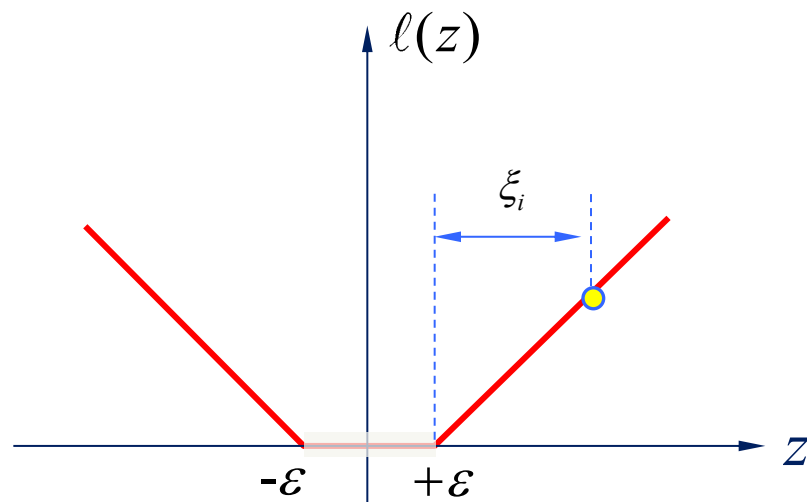
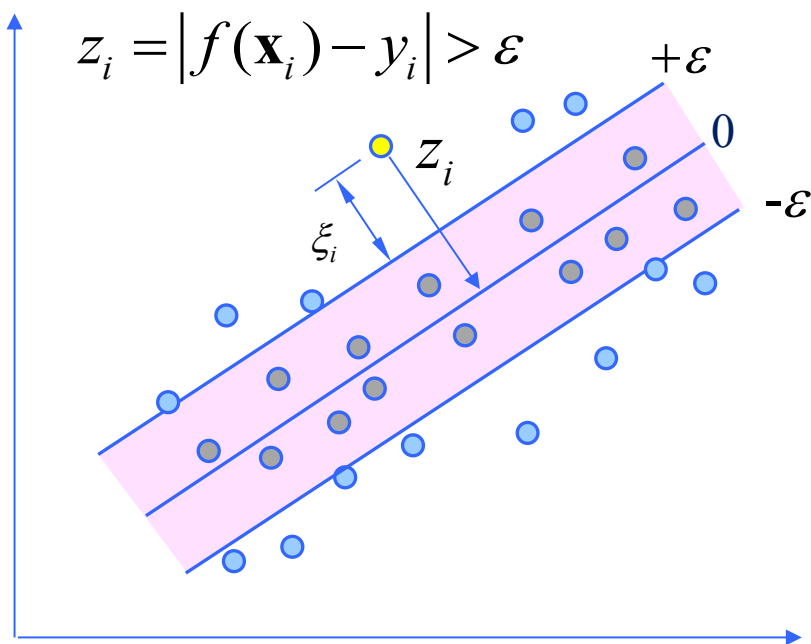
ε -insensitive loss function
 ε -不敏感损失函数

$$\ell_{\varepsilon}(z) = \begin{cases} 0, & \text{if } |z| \leq \varepsilon \\ |z| - \varepsilon, & \text{otherwise} \end{cases}$$



15.11.2 支持向量机回归

- 学习模型
$$\min_{\mathbf{w}, \mathbf{b}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \ell_{\varepsilon}(f(\mathbf{x}_i) - y_i),$$



$$\ell_{\varepsilon}(z) = \begin{cases} 0, & \text{if } |z| \leq \varepsilon \\ |z| - \varepsilon, & \text{otherwise} \end{cases}$$

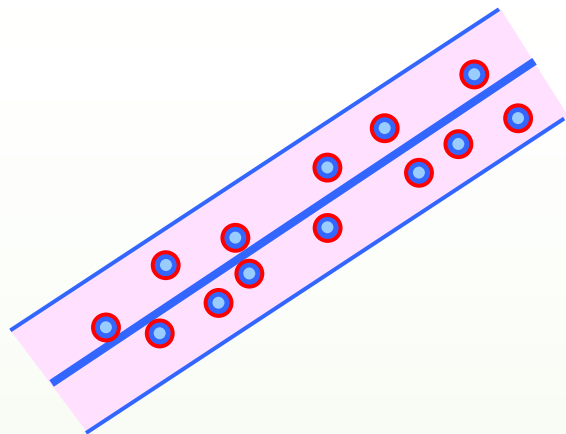
ε -不敏感损失函数

对应的损失为: $|f(\mathbf{x}_i) - y_i| - \varepsilon$

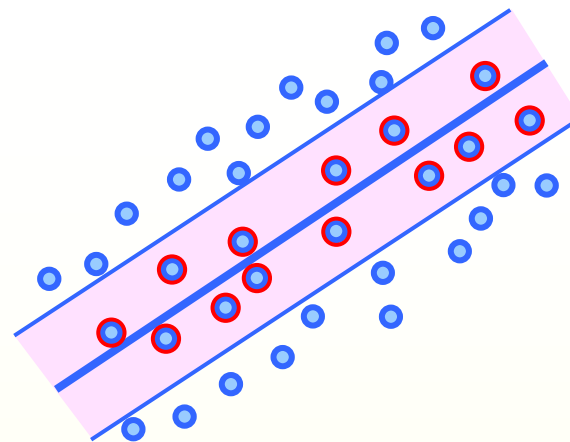
15.11.2 支持向量机回归

- 学习模型（从支持向量机的角度）

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$



$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 \\ \text{s.t.} \quad & |f(\mathbf{x}_i) - y_i| \leq \varepsilon, \\ & i = 1, 2, \dots, n \end{aligned}$$



$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \xi_i \\ \text{s.t.} \quad & |f(\mathbf{x}_i) - y_i| - \xi_i \leq \varepsilon, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

15.11.2 支持向量机回归

学习模型:

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2$$

$$s.t. \quad |f(\mathbf{x}_i) - y_i| - \xi_i \leq \varepsilon,$$

$$\xi_i \geq 0,$$

$$i = 1, 2, \dots, n$$



$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n \xi_i$$

$$s.t. \quad -\varepsilon - \xi_i \leq f(\mathbf{x}_i) - y_i \leq \varepsilon + \xi_i,$$

$$\xi_i \geq 0,$$

$$i = 1, 2, \dots, n$$

15.11.2 支持向量机回归

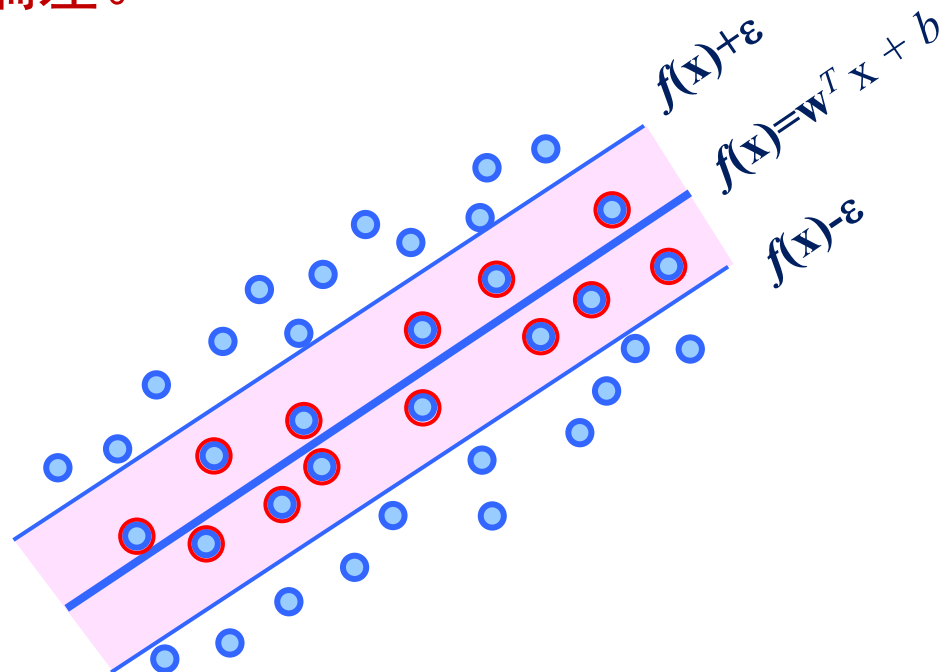
学习模型（松弛模型）

$$\begin{aligned} \min_{\mathbf{w}, b, \xi_i, \hat{\xi}_i} \quad & \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^n (\xi_i + \hat{\xi}_i) \\ \text{s.t.} \quad & f(\mathbf{x}_i) - y_i \leq \varepsilon + \xi_i, \\ & y_i - f(\mathbf{x}_i) \leq \varepsilon + \hat{\xi}_i, \\ & \xi_i \geq 0, \\ & \hat{\xi}_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

Similar to SVM, this can be solved as a quadratic programming problem

15.11.2 支持向量机回归

- Support vector regression, SVR: 假设 $f(\mathbf{x})$ 与 y 之间可以有 ε 容忍偏差。



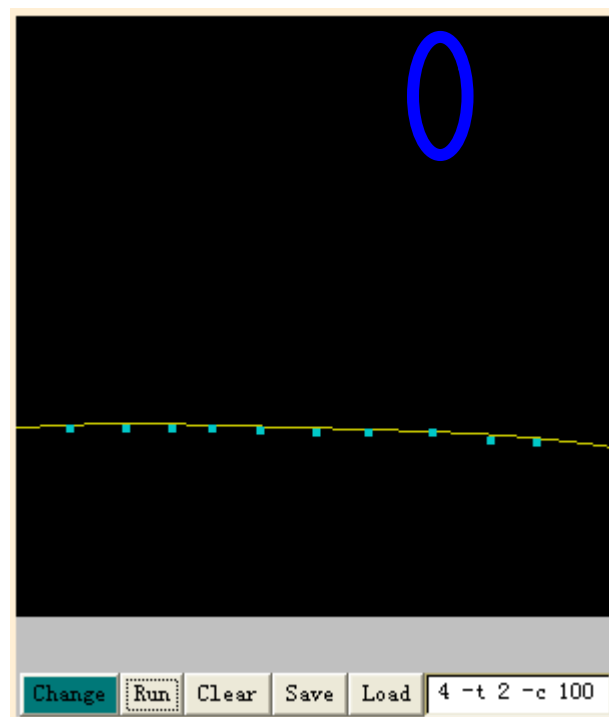
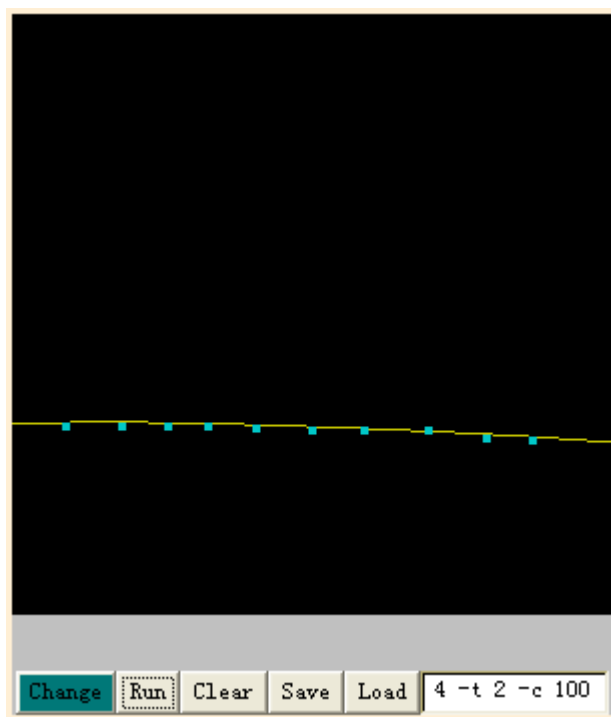
- ✓ 对偶优化问题中拉格朗日乘数非零的样本，直接决定了 ε -不敏感带的位置、宽度以及回归超平面的参数。

在所有样本点中，只有分布在“管壁”和“管壁”之外的样本点决定管道的位置。这一部分训练样本称为“支持向量”。

15.11.2 支持向量机回归

Demo

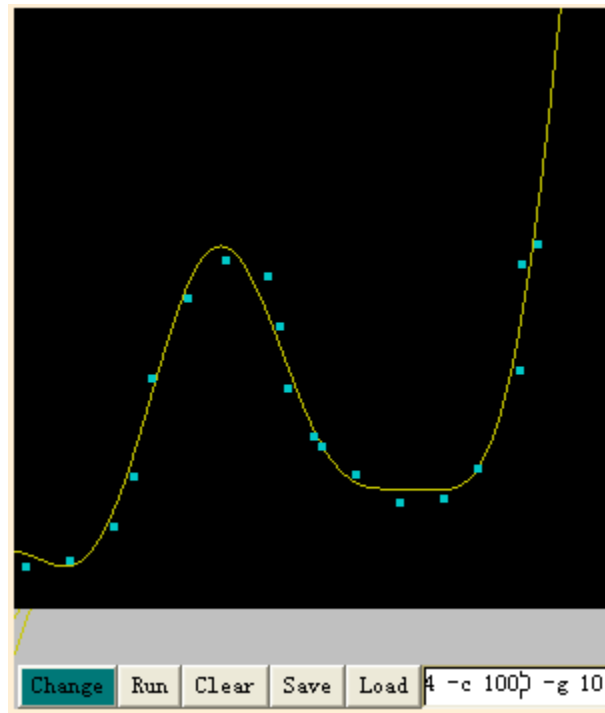
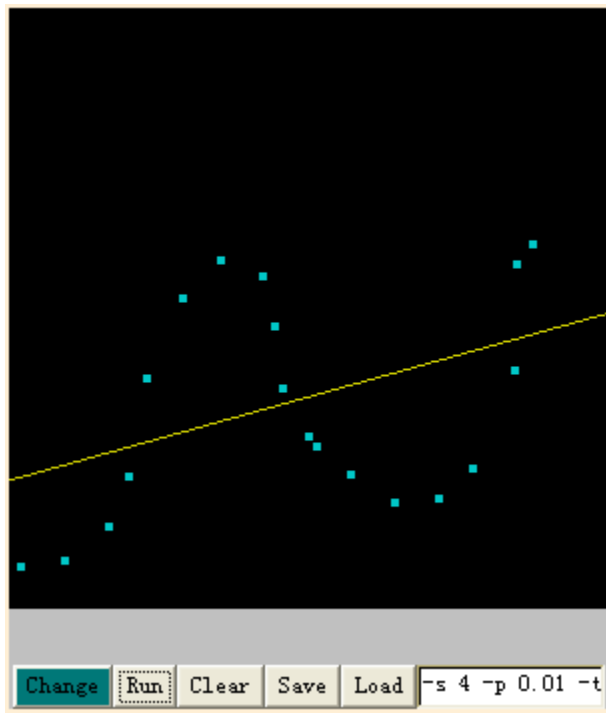
- Less Sensitive to Outlier



15.11.2 支持向量机回归

Again, Extend to Non-Linear Case

- Similar with SVM



15.12 SVM for Ranking

15.12 支持向量机排序

- **Learning to rank (L2R)**

- 排序一直是信息检索的核心问题之一，Learning to Rank(简称LTR)用机器学习的思想来解决排序问题。
- L2R 有三种主要的方法：PointWise，PairWise，ListWise。
- Ranking SVM 算法是 PointWise 方法的一种，由 R. Herbrich 等人在2000提出。
 - RankSVM的基本思想是，将排序问题转化为pairwise的分类问题，然后使用SVM分类模型进行学习并求解。
 - SVM Ranking 的核心原理：从“点预测”到“序约束，约束条件从“分类边界”或“ ϵ -带”转化为“序关系满足”。

15.12 支持向量机排序

- 将排序问题转化为分类问题

- 比如，以文档查询为背景 “query-doc pair”。
- 记一个文档的特征为 $\mathbf{x}$ ，我们的目的是需要找到一个排序函数 $f(\mathbf{x})$ ，根据 $f(\mathbf{x})$ 的大小来决定排序顺序。即如果 $f(\mathbf{x}_i) > f(\mathbf{x}_j)$ ，则 $\mathbf{x}_i$ 应该排在 $\mathbf{x}_j$ 的前面，反之亦然：

$$\mathbf{x}_i \succ \mathbf{x}_j \iff f(\mathbf{x}_i) > f(\mathbf{x}_j)$$

- 理论上， $f(\mathbf{x})$ 可以是任意函数。
- 为了简单起见，假设其为线性函数： $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$
- 由于排序不受参数 b 的影响，所以可以令： $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$

15.12 支持向量机排序

- 为什么可以转换为两类分类问题？

- 首先，对于任意两个数据点 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ ，若 $f(\mathbf{x})$ 是线性函数，则如下关系成立：

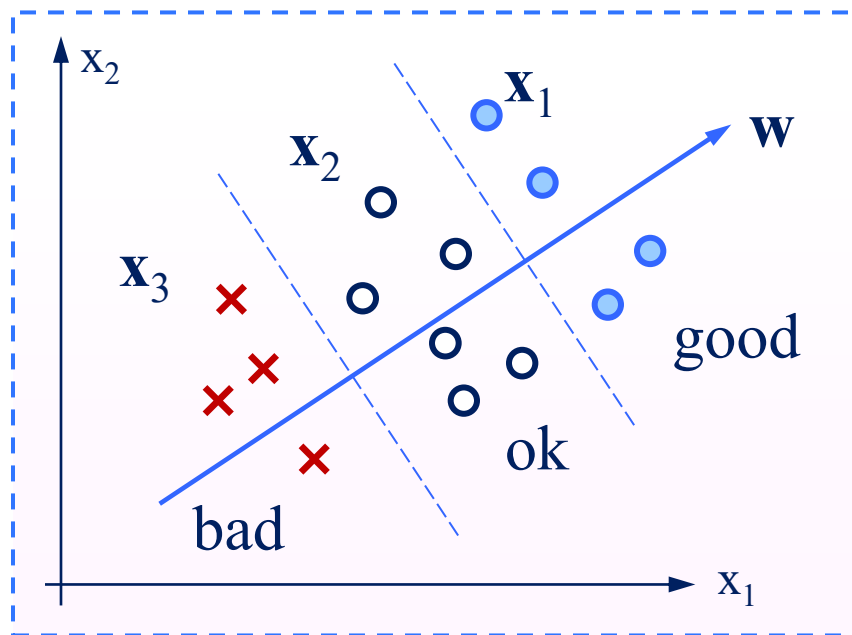
$$f(\mathbf{x}_i) > f(\mathbf{x}_j) \iff \mathbf{w}^T (\mathbf{x}_i - \mathbf{x}_j) > 0$$

- 然后，可以对 $\mathbf{x}_i$ 和 $\mathbf{x}_j$ 的差值向量引入两类分类问题，按如下方式进行标签赋值：

$$y = \begin{cases} +1, & \text{if } x_i \succ x_j \\ -1, & \text{if } x_i \prec x_j \end{cases}, \quad \mathbf{w}^T (\mathbf{x}_i - \mathbf{x}_j) > 0 \iff y = +1$$

• SVM模型解决排序问题

- 将排序问题转化为分类问题之后，可使用Linear SVM或 kernel SVM解决排序问题。



上图展示了一组查询，给出了所召回的文档，其中文档的相关程度等级分为三档(good, ok, bad)。权重向量 w 对应排序函数，可以对“查询-返回”对进行打分和排序。

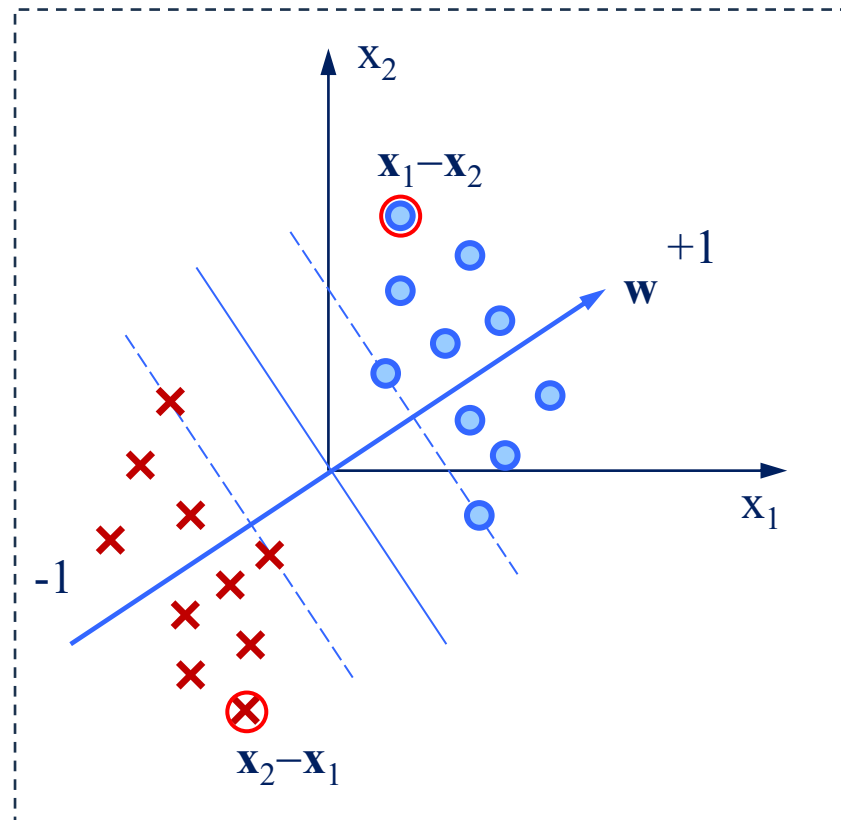
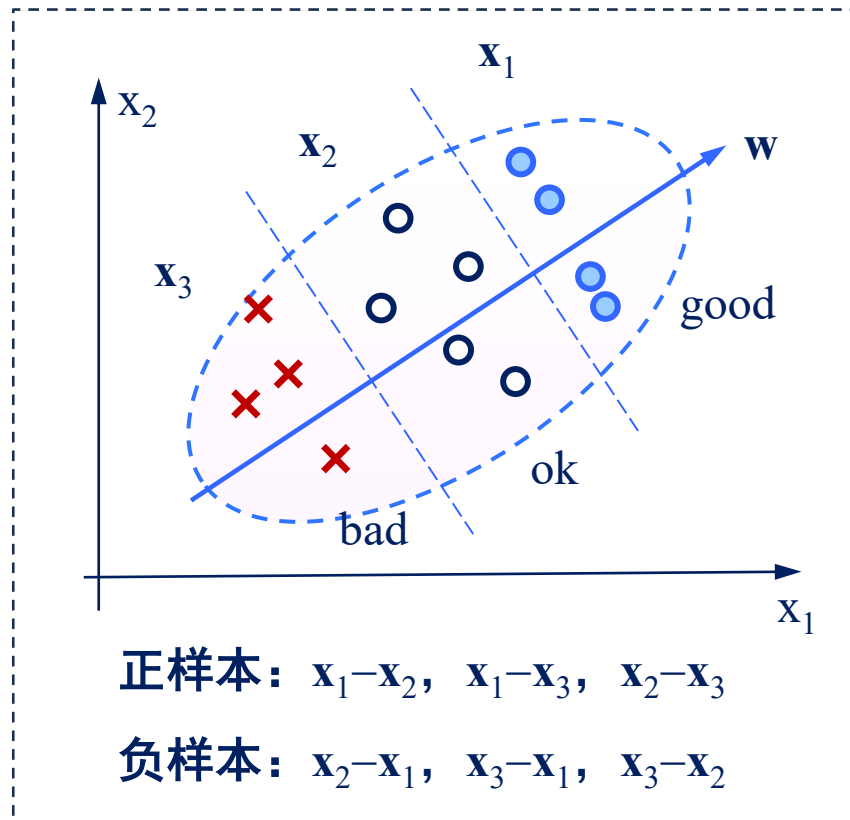
15.12 支持向量机排序

为SVM准备样本：

- ✓ 给定一个查询及其反馈，可对样本进行组合，形成新数据点： $\mathbf{x}_1 - \mathbf{x}_2$ ， $\mathbf{x}_1 - \mathbf{x}_3$ ， $\mathbf{x}_2 - \mathbf{x}_3$ 。其label也会被重新赋值，比如将 $\mathbf{x}_1 - \mathbf{x}_2$ ， $\mathbf{x}_1 - \mathbf{x}_3$ ， $\mathbf{x}_2 - \mathbf{x}_3$ 的label赋值为正类。
- ✓ 为了构造分类问题，还需负样本。可以使用其反方向向量作为负样本： $\mathbf{x}_2 - \mathbf{x}_1$ ， $\mathbf{x}_3 - \mathbf{x}_1$ ， $\mathbf{x}_3 - \mathbf{x}_2$ 。
- ✓ 需要指出的是，在组合形成新样本时，不能使用在原始排序问题中处于相同相似度等级的两个数据点，也不能使用处于不同query下的两个数据点来组合新样本。

15.12 支持向量机排序

为SVM准备样本：



15.12 支持向量机排序

- 学习模型

- 转化为分类问题后，便可以采用SVM的通用方式进行求解。学习模型如下：

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\begin{aligned} s.t. \quad & y_i (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

支持向量机分类

$$\min_{\mathbf{w}, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

$$\begin{aligned} s.t. \quad & y_i \left(\mathbf{w}^T \left(\mathbf{x}_i^{(1)} - \mathbf{x}_i^{(2)} \right) \right) \geq 1 - \xi_i, \\ & \xi_i \geq 0, \\ & i = 1, 2, \dots, n \end{aligned}$$

支持向量机排序

15.12 支持向量机排序

- 学习模型

- 类似地，采用合页损失函数：

$$\min_{\mathbf{w}, b} \sum_{i=1}^n \left[1 - y_i \left(\mathbf{w}^T \left(\mathbf{x}_i^{(1)} - \mathbf{x}_i^{(2)} \right) \right) \right]_+ + \lambda \|\mathbf{w}\|_2^2$$

$$\text{where } [z]_+ = \begin{cases} z, & \text{if } z > 0 \\ 0, & \text{otherwise} \end{cases}$$

15.12 支持向量机排序

- 对偶学习模型

支持向量机分类

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j (\mathbf{x}_i \cdot \mathbf{x}_j) - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0; \\ & 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, n \end{aligned}$$



$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \left(\mathbf{x}_i^{(1)} - \mathbf{x}_i^{(2)} \right) \cdot \left(\mathbf{x}_j^{(1)} - \mathbf{x}_j^{(2)} \right) - \sum_{i=1}^n \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^n \alpha_i y_i = 0; \\ & 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, n \end{aligned}$$

支持向量机排序

15.12 支持向量机排序

- 其它改进

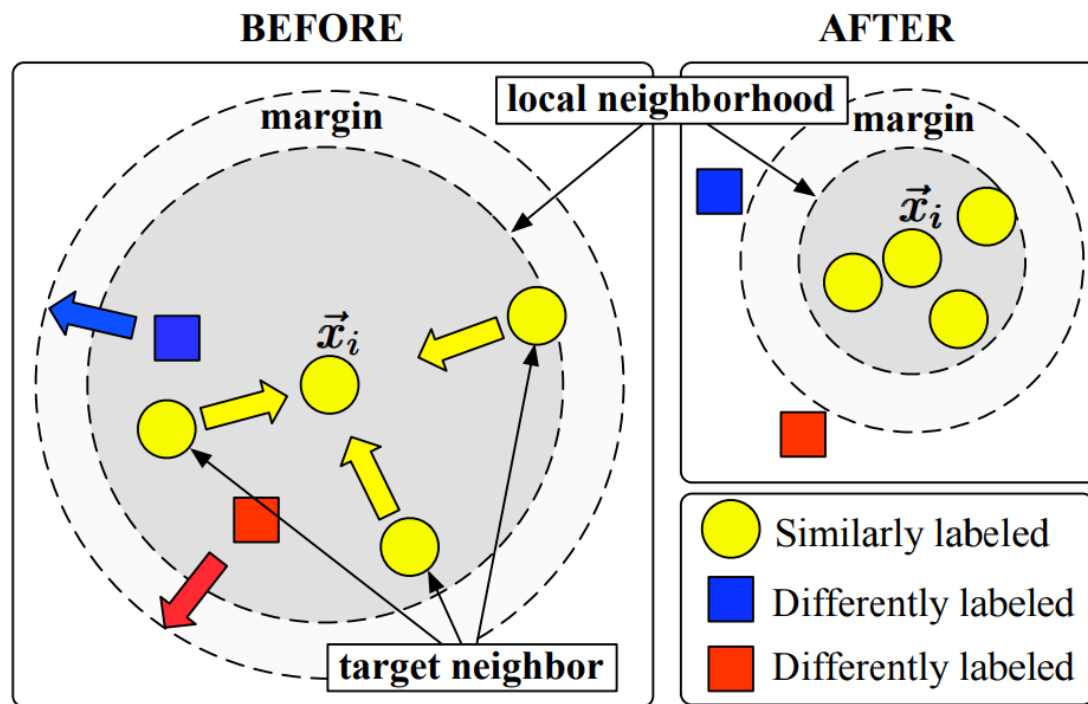
- Hinge Loss 是 0-1 损失。损失函数的优化目标可以进一步与信息检索的 Evaluation 常用指标建立紧密联系。
- **更复杂的方法：**采用 Ordinal Regression 方法来对此问题进行建模。
- 还要可以将对偶问题转化为核学习技巧。

15.13 Large Margin Nearest Neighbor (LMNN)

15.13 Large Margin Nearest Neighbor (LMNN)

- Learn a Mahalanobis distance metric in the kNN classification setting by semi-definite programming (SDP), that
 - Enforce the k-nearest neighbors within the same class more closely
 - Examples from different classes are separated by a large margin

(Weinberger et al., 2006)



After training, we hope:

- (1) $k=3$ target neighbors lie within a smaller radius
- (2) differently labeled inputs lie outside this smaller radius with a margin of **at least one unit distance**

学习模型

属于同一类的邻近点之间的距离要小

同一邻域内，不属于同一类的点对

$$\min \sum_{ij} \eta_{ij} (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{M} (\mathbf{x}_i - \mathbf{x}_j) + \sum_{ijl} \eta_{ij} (1 - y_{il}) \xi_{ijl}$$

$$s.t. \quad \frac{(\mathbf{x}_i - \mathbf{x}_l)^T \mathbf{M} (\mathbf{x}_i - \mathbf{x}_l) - (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{M} (\mathbf{x}_i - \mathbf{x}_j)}{2} \geq 1 - \xi_{ijl}$$

$$\xi_{ijl} \geq 0$$

$$\mathbf{M} \succcurlyeq 0$$

同一邻域内，不属于同一类的点之间的距离要大于“1”

relax variables

$$\eta_{ij} = \begin{cases} 1, & \text{if } (\mathbf{x}_i, \mathbf{x}_j) \in \text{the same class, and } \mathbf{x}_j \in N_k(\mathbf{x}_i) \\ 0, & \text{otherwise} \end{cases}$$

$$y_{il} = \begin{cases} 1, & \text{if } (\mathbf{x}_i, \mathbf{x}_l) \in \text{the same class} \\ 0, & \text{otherwise} \end{cases}$$

Kilian Q. Weinberger, John Blitzer and Lawrence K. Saul. Distance Metric Learning for Large Margin Nearest Neighbor Classification, NIPS, 2005.

15.14 Kernel Learning (Extension)

15.14.1 核主成分分析: KPCA

- PCA

- 给定 $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n] \in \mathbb{R}^{n \times m}$, 并假定均值为零
- 计算协方差矩阵: $\mathbf{C} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \mathbf{x}_i^T$
- 对 $\mathbf{C}$ 施行矩阵特征值分解: $\mathbf{C} = \mathbf{U} \mathbf{\Sigma} \mathbf{U}^T$
- 取**指定个数**的最大特征值对应的特征向量子集, 组成投影向量 $\mathbf{W} = \mathbf{U}_s$, ($\mathbf{U}_s$ 为 $\mathbf{U}$ 的子矩阵)
- 对新样本 $\mathbf{x}$, 将其投影至低维子空间: $\mathbf{y} = \mathbf{W}^T \mathbf{x}$

- 核主成分分析：KPCA

- 引入非线性映射： $\phi: \mathbb{R}^m \rightarrow F$

- 将数据进行映射： $\{\phi(\mathbf{x}_1), \phi(\mathbf{x}_2), \dots, \phi(\mathbf{x}_n)\} \subset F$

- 计算协方差矩阵： $\bar{\mathbf{C}} = \frac{1}{n} \sum_{i=1}^n \phi(\mathbf{x}_i) \phi(\mathbf{x}_i)^T$

- 作特征值分解： $\bar{\mathbf{C}} = \bar{\mathbf{U}} \bar{\mathbf{\Sigma}} \bar{\mathbf{U}}^T$

- What are the difficulties? (如何实现)

- B. Scholkopf, A. J. Smola, K. R. Muller. Nonlinear Component Analysis as a Kernel Eigenvalue Problem. Neural Computation, 10(5):1299–1319, 1998.

• 主要结论：与PCA作对比

– 在PCA中， $\mathbf{W}$ 为样本协方差矩阵的前 d 个特征值对应的特征向量所构成： $\mathbf{W}=[\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_d] \in R^{m \times d}$ ，

• 样本点 $\mathbf{x}$ 的投影： $\mathbf{y} = \mathbf{W}^T \mathbf{x} = [\mathbf{w}_1 \cdot \mathbf{x}, \mathbf{w}_2 \cdot \mathbf{x}, \dots, \mathbf{w}_d \cdot \mathbf{x}]^T \in R^d$

– 在KPCA中， $\mathbf{W}$ 为样本在高维特征空间中的协方差矩阵的前 d 个特征值对应的特征向量所构成：

$$\mathbf{W} = \left[\mathbf{v}_1 = \sum_{i=1}^n \alpha_i^1 \phi(\mathbf{x}_i), \mathbf{v}_2 = \sum_{i=1}^n \alpha_i^2 \phi(\mathbf{x}_i), \dots, \mathbf{v}_d = \sum_{i=1}^n \alpha_i^d \phi(\mathbf{x}_i) \right]$$

• 样本点 $\mathbf{x}$ 的投影：

$$\begin{aligned} \mathbf{y} &= \left[\mathbf{v}_1^T \phi(\mathbf{x}), \mathbf{v}_2^T \phi(\mathbf{x}), \dots, \mathbf{v}_d^T \phi(\mathbf{x}) \right]^T \\ &= \left[\sum_{i=1}^n \alpha_i^1 K(\mathbf{x}_i, \mathbf{x}), \sum_{i=1}^n \alpha_i^2 K(\mathbf{x}_i, \mathbf{x}), \dots, \sum_{i=1}^n \alpha_i^d K(\mathbf{x}_i, \mathbf{x}) \right]^T \end{aligned}$$

由数据点构成的核矩阵的第一个（上标）特征向量第 i 个（下标）分量

15.14.2 关于核化的一般性理论

- 主要参考文献

- Changshui Zhang, Feiping Nie, Shiming Xiang: *A general kernelization framework for learning algorithms based on kernel PCA*. Neurocomputing 73(4-6): 959-967, 2010

15.14.2 关于核化的一般性理论

- **满秩PCA**

- 对于训练数据 $\mathbf{X}$ ，设其中心化的内积矩阵（即协方差矩阵） $\mathbf{C}$ 的秩为 r ，如果提取PCA的前 r 个主成分，则称此过程为满秩PCA。

- **满秩KPCA**

- 对于训练数据 $\mathbf{X}$ ，设其中心化的核矩阵 $\mathbf{K}$ 的秩为 r ，如果提取KPCA的前 r 个主成分，则称此过程为满秩KPCA。

15.14.2 关于核化的一般性理论

- 定理:

如果一个线性算法同时满足如下两个条件:

- (1) 算法的输出仅与内积运算 $\mathbf{x} \cdot \mathbf{x}_i$, ($i = 1, 2, \dots, n$) 有关;
- (2) 对训练数据的平移不会改变算法的输出结果;

则该算法的核化可以通过对数据先做满秩KPCA变换, 然后在变换后的数据上直接再做该线算法来实现。

15.14.2 关于核化的一般性理论

- 举例
 - $\text{KSVM} = \text{KPCA} + \text{SVM}$
 - $\text{KLDA} = \text{KPCA} + \text{LDA}$
 - $\text{KCCA} = \text{KPCA} + \text{CCA}$
 - $\text{KPLS} = \text{KPCA} + \text{PLS}$
 - 核岭回归 = $\text{KPCA} + \text{岭回归}$

在实际应用中，对有噪声的数据，采用低秩PCA会更好！

15.15 Conclusion

- Structural Risk Minimization → VC Dimension
- Large Margin → Low VC Dimension
- Hard Margin SVM → Soft Margin SVM
- Dual Problem → Kernel Method
- Model Selection → Tradeoff C and Kernel
- Optimization → SMO
- Multi-Class SVM → Binary or Single-Machine
- Least Squares Extensions → Another Perspective
- PCA → KPCA
- Large Margin for Regression, Ranking, Nearest Neighbors
- 关于核学习的一般性结论
- Ideas of: **large margin, hinge loss, kernel method**, are widely used in different PR & ML tasks

致谢：本讲部分PPT由张煦尧研究员提供

Thank All of You!
(Questions?)

向世明

smxiang@nlpr.ia.ac.cn

people.ucas.ac.cn/~xiangshiming

时空数据分析与学习课题组(STDAL)

中科院自动化研究所·模式识别国家重点实验室